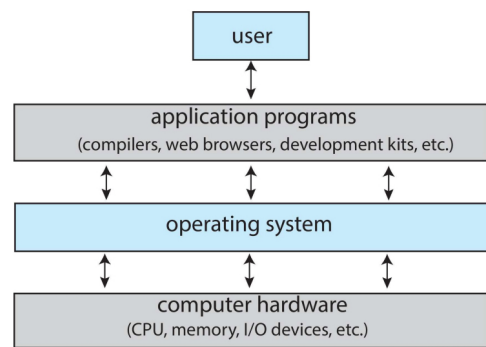


## 2주차

- 목표 : 컴퓨터 시스템의 구조와 Interrupt의 역할  
Multiprocessor 시스템의 구성 요소  
User mode와 Kernel mode로의 전환  
다양한 컴퓨팅 환경에서의 OS 활용



### 컴퓨터 시스템의 4가지 component

1. **하드웨어** : 기본적인 계산 리소스 제공  
ex) CPU, 메모리, I/O 장치
2. **OS** : 다양한 사용자들을 위해 여러 응용프로그램 간 **하드웨어 사용을 제어, 조정**
3. **응용 프로그램** : 사용자의 컴퓨팅 문제를 해결하기 위해 **시스템 자원을 사용하는 방식 정의**
4. **사용자(user)** : 사람, 기계, 다른 컴퓨터 등...

### 관점에 따라 다름

#### User View 유저 관점

유저는 편하고, 쓰기 쉽고, 성능 좋은 것을 원함

자원 활용에는 관심이 없음.

어떤 컴퓨터(임베디드/자동차)는 user 인터페이스가 없는 경우도 있고, 사람의 개입 없이 스스로 돌아가기도 함

#### System View 시스템 관점

OS는 **자원 할당, 제어 프로그램** 역할. 프로세스 간 충돌하는지 등등 감시

### OS의 역할

우리 주변에 어디에는 있음. 에어컨 공기청정기 등...

군사 목적 -> **범용 장치**로 변화해 옴

컴퓨터가 범용 장치가 되면서 한정된 자원을 관리할 **관리자**가 필요해짐.

### OS의 정의

OS를 한마디로 정의하기는 어려움.

OS를 주문했을 때 제조사가 보내 주는 모든 것. 윈도우10을 사면 그 안에 들어 있는 것

### Kernel 커널

: **OS의 핵심이 되는 부분**. 컴퓨터에서 **항상 실행**되고 있는 하나의 프로그램. 쉬지않고 동작함. OS의 일부분.

세 가지 목적 : **편의성, 효율성, 진화 능력**(Convenience, Efficiency, Ability of evolve)

컴퓨터에서 동작하는 제일 중요한 소프트웨어

Kernel을 제외한 모든 것은 System Program(프린터 제어 등)이거나 Application Program이다.

### 컴퓨터 시스템의 구성 : 4가지

#### 프로세서

컴퓨터의 동작을 제어. 데이터 처리 기능 수행. Central Processing Unit

#### 메인 메모리

데이터와 프로그램을 저장. 일반적으로 Volatile 휘발성(전원 꺼지면 내용 사라짐)

aka 실제 메모리(Real memory), 주 메모리(Primary Memory)

#### I/O 모듈

컴퓨터와 외부 환경(외부 장치) 사이에서 데이터를 이동시킴. ex) 통신장비(랜카드 등..), 단말(마우스 등..)

I/O 장치와 CPU는 concurrent하게(독립적으로) 작동 할 수 있다. Time-Sharing 시분할. Idle 시간 줄임.

## 시스템 버스 System bus

프로세스, 주기억장치, I/O 모듈 간 통신을 제공. 각 요소 들간의 “길” or “연결 통로”

## 레지스터 종류

PC : Program Counter 다음에 실행될 명령어 주소 보관

IR : Instruction Reg. 현재 실행할 명령어 가지고 있음.

MAR : Memory Address Reg. 읽거나 쓰고자 하는 메모리의 주소 저장

MBR : Memory Buffer Reg. 메모리로부터 읽어온 데이터나 메모리에 쓸 데이터를 임시로 저장

I/O AR : I/O Address Reg. 접근하려는 특정 입출력 장치의 주소를 저장.

I/O BR : I/O Buffer Reg. 입출력 장치와 주고받는 데이터를 임시로 저장

## 명령어 실행

“프로그램”은 메모리에 저장된 일련의 instruction들로 구성된다.

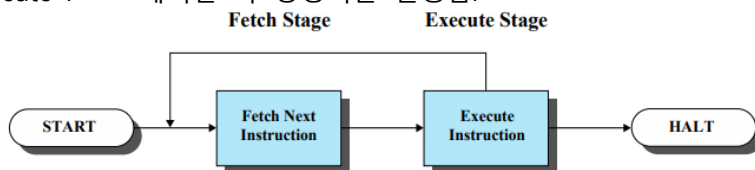
Fetch -> Execute 2단계로 나누어짐

Fetch : 프로세서가 메모리로부터 명령어를 읽어 옴

PC가 알려준 주소로 가서 명령어를 읽어 옴.

읽어 온 메모리는 IR에 저장됨

Execute : 프로세서는 각 명령어를 실행함.



프로그램이 시작되면, instruction을 메모리로부터 fetch하고 가져온 instruction을 실행함. 끝날 때까지 반복

## Interrupts 인터럽트 ★★★

OS는 Interrupt 기반으로 작동됨

I/O나 메모리와 같은 다른 모듈이 프로세서의 정상적인 수행 순서를 중단.

프로세서의 활용도를 높이기 위해서 Interrupt를 사용.

대부분의 I/O 디바이스는 프로세서보다 느림.

그래서 프로세서는 장치가 동작을 마칠 때까지 기다려야 함. => 낭비다!

## Interrupt의 종류 - 4가지

### Program 프로그램

산술 오버플로우, 0으로 나누기, 잘못된 명령어 실행, 유저에게 허용된 메모리 범위 이외 접근 등 명령어 실행 결과로 인해서 발생함.

프로그램이 강제로 종료될 수 있음.

### Timer 타이머

프로세스 내의 타이머에 의해서 생성됨. 운영체제가 정기적으로 특정 기능을 수행할 수 있게 함.

ex) 특정 시간마다 시스템을 점검하거나 다른 프로세스로 교체할 때 사용됨

## I/O 입출력

I/O 컨트롤러가 작업의 **정상적인 완료**를 알려거나 오류가 발생했을 때 신호를 보냄

### Hardware failure

전원 공급 중단, 메모리 오류같은 고장이 났을 때.

ex) 정전, 메모리 고장으로 블루스크린 시

### Interrupt 그림

#### (a) 인터럽트 없음

I/O를 할 때 CPU가 아무것도 안 하고 기다리고 있음.

#### (b) 짧은 I/O Interrupt

CPU는 I/O를 던져놓고 바로 2a로 돌아와서 할 일을 하다가 I/O할 할일을 끝내면 "Interrupt Handler"를 호출해서 CPU를 잠시 멈춤 -> CPU는 쉬지 않는다.

#### (c) 긴 I/O Interrupt

ex) 대용량 저장을 할 때

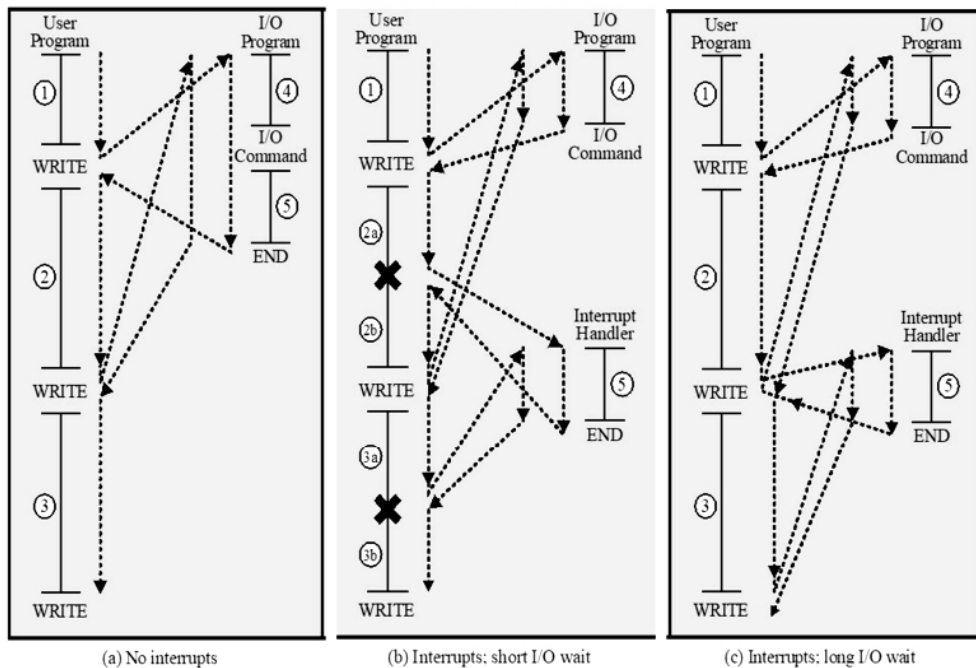
1. 첫 번째 I/O 시작 : WRITE를 만나서 4를 실행함.

2. CPU 복귀 : 다시 CPU가 유저 프로그램으로 복귀

3. 장치 충돌 및 대기 : 유저 프로그램이 다음 WRITE를 만남. but, 첫번째 I/O작업이 아직 수행중이라서 장치 사용 권한이 없음. **WAIT** 상태로 대기.

4. 인터럽트 발생 및 다음 작업(I/O 시작) : 첫 번째 작업이 **완료되어** Interrupt가 걸리면(그림에서 5번) 두 번째 WRITE I/O Interrupt를 수행함(그림에서 동글 4번)

5. 다시 돌아와서 유저 프로그램을 실행...

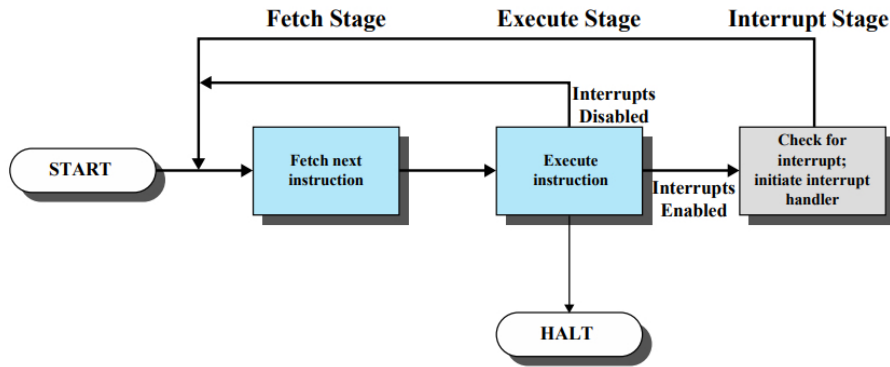


### CPU vs I/O 장치

CPU : 1GHz ==  $10^9$  1초에 10억개 명령어 처리 가능

디스크 탐색 : 7200rpm 디스크의 경우 찾는 데 4.17ms가 걸림

=>  $\frac{4 \times 10^{-3}}{10^{-9}} = 4 \times 10^6$  ∴ I/O 하는 동안 400만개 명령어 실행 가능



Instruction 완료하고 난 후 무조건 interrupt가 있는지 체크. Flag를 체크해야 함.  
있다면 Interrupt Handler를 호출한다.

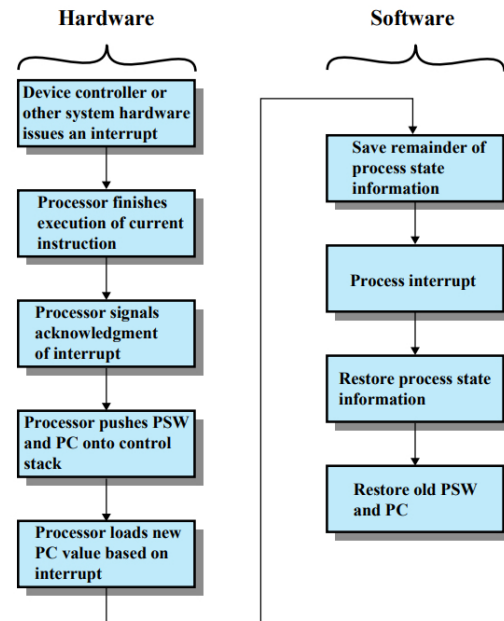
## 간단한 Interrupt 프로세싱

### 하드웨어

- 장치 컨트롤러나 다른 시스템 HW가 인터럽트 발생.
- > 현재 실행중인 instruction을 완료함
- > 프로세서가 인터럽트 승인 신호를 보냄
- > 프로세서가 PSW와 PC 제어 스택에 Push함  
(현재 상태 저장)
- > 프로세서가 인터럽트에 기반한 새 PC 값 로드

### 소프트웨어

- 프로세스 상태 정보의 남아 있는것을 저장  
(HW에서 한 것 이외에 SW에서도 저장)
- > 프로세스 Interrupt
- > 프로세스 상태 정보를 복구. 백업한거 불러옴
- > 이전 PSW와 PC를 복구함.  
(인터럽트 걸리기 전 상태로 돌아가기)



\* **PSW** : Program Status Word 지금 CPU에 들어있는 프로그램에 대한 정보.  
작업 모드(유저?커널?), 메모리 정보(메모리 어느 영역?), 기타 상태...

## 다중 인터럽트 Multiple Interrupts

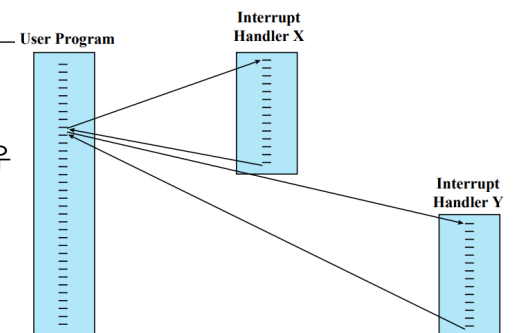
하나의 인터럽트를 처리하는 중에 또 다른 인터럽트가 발생하는 상황  
ex) 통신 회선에서 데이터를 받으면서 동시에 결과를 인쇄하는 경우

### 두 가지 접근방식

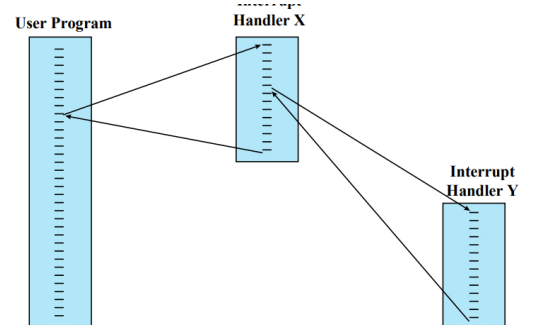
- 1) 인터럽트 처리 중에는 다른 인터럽트가 발생하지 못하도록 하기
- 2) 우선순위(Priority) 체계 사용

순차적 인터럽트 처리 Sequential Interrupt Processing  
: 발생한 인터럽트를 순서대로 하나씩 처리하는 방식

중첩된 인터럽트 처리 Nested Interrupt Processing  
: 인터럽트 처리 중에 더 중요한 인터럽트가 발생하면,  
현재 작업을 멈추고 새 인터럽트를 처리.  
Priority에 따라서 처리!



(a) Sequential interrupt processing



(b) Nested interrupt processing



## 스토리지 구조

**Main Memory 메인 메모리** : CPU가 직접 액세스 할 수 있는 유일한 대용량 저장매체

일반적으로 **volatile 휘발성**임.

일반적으로 **DRAM** 형태의 **Random Access Memory** 랜덤 액세스 메모리임 => 주소 알면 즉시 접근 가능

**Secondary Memory 보조저장장치** : 메인 메모리의 용량 한계를 보완. 전원이 꺼져도 데이터 유지. Non-volatile

HDD(Hard Disk Drive) : 자기 기록 물질로 덮인 단단한 금속 또는 유리 플레이트

NVM(Non-Volatile Memory) 장치 : 하드 디스크보다 빠르고 비휘발성. ex) SSD, 플래시 메모리

최근에 가가격 하락, 대중화됨.

## 컴퓨터 시작 Computer Startup

**Bootstrap Program 부트스트랩 프로그램** : 전원을 켜거나 재부팅할 때 로드됨.

**EEPROM**에 저장됨. aka 펌웨어 firmware (Electrically Erasable Programmable Read-Only Memory)

시스템의 모든 측면을 초기화

운영체제 커널을 메모리에 로드하고 실행을 시작.

## +용량 정의와 표기

3PB = 300PB

10<sup>3</sup> 단위로 계산

Petabyte

3PiB = 3072TB

실제 1024단위(2<sup>10</sup>)으로 계산

Pebibyte

~bibyte

## 스토리지 계층 Storage Hierarchy

속도 비용 휘발성이 다 다름

계층이 내려갈수록...

비트당 비용 감소 - 저렴해짐

용량이 커짐

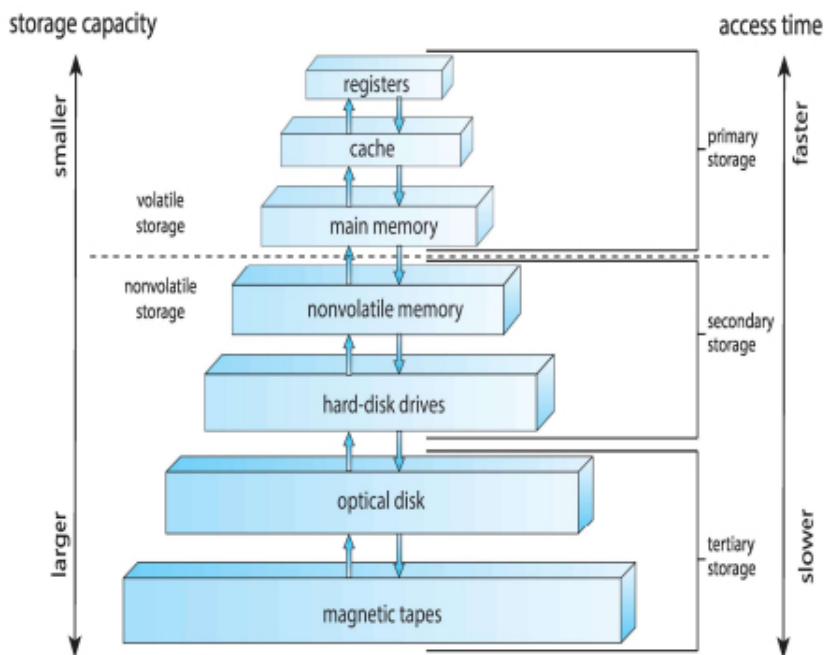
접근 시간이 증가됨

프로세서의 메모리 접근 빈도 감소

레지스터와 캐시는 CPU 내부에 위치

|                 |      |
|-----------------|------|
| Non-volatile    | SSD  |
| Hard-Disk Drive | HDD  |
| Optical Disk    | CD   |
| Magnetic tapes  | Tape |

해킹위험X, 영구보존용 but Random-Access 불가능! 접근 시간 매우 느림



## 직접 메모리 접근 구조 Direct Memory Access Structure

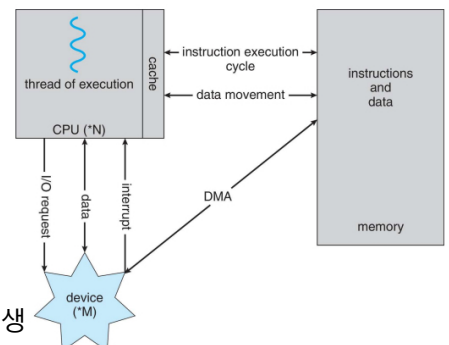
: 메모리 속도에 가깝게 정보를 전송할 수 있는 고속 I/O 장치에 사용

HDD나 네트워크 카드처럼 데이터를 많고, 빠르게 주고받을 때 사용

device 컨트롤러가 CPU 개입 없이 버퍼 저장소에서 메인 메모리로

데이터 블록을 직접 전송

바이트당 1번의 인터럽트가 발생하는 대신, 블록당 단 1번의 인터럽트만 발생



## 컴퓨터 시스템 아키텍처 Computer System Architecture

대부분의 시스템은 하나의 general-purpose 범용 프로세서를 사용  
대부분의 시스템에는 special-purpose 특수 프로세서도 있음.  
-> 멀티프로세서 시스템의 사용과 중요성이 증가하고 있음.

장점 : 1. Throughput 처리량 증가  
2. Increased readability

### Asymmetric Multiprocessing 비대칭형 멀티프로세싱

각 프로세서에 특정 작업이 할당되는 방식 -> 거의 사용되지 않음

### Symmetric Multiprocessing 대칭형 멀티프로세싱

각각의 프로세서가 모든 작업을 나누어 수행. 메인 메모리 공유.

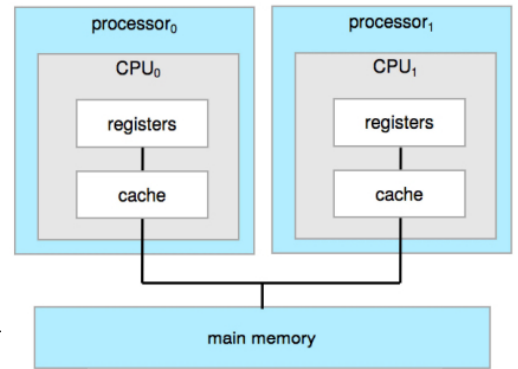


Figure 1.8 Symmetric multiprocessing architecture.

## 듀얼 코어 설계 Dual-Core Design

멀티프로세서가 발전함 -> 한 칩 안에 여러 코어를 넣는 방식으로 진화.

on-chip(온칩) 통신이 칩 간 통신보다 빠르기 때문에 멀티코어 시스템이 더 효율적일 수 있음.

멀티코어 칩은 전력을 훨씬 적게 소모. 모바일 디바이스, 노트북에 중요함.

## NUMA Non-Uniform Memory Access System

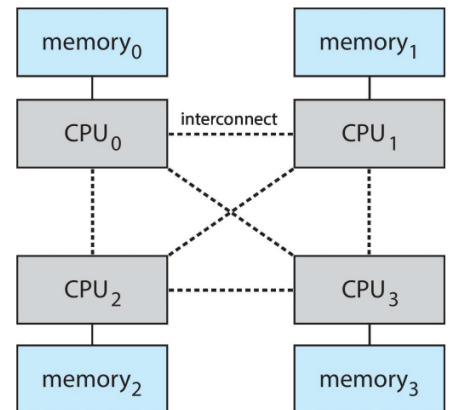
CPU가 엄청나게 많아지면, 메인 메모리를 공유하는데 경쟁이 발생.

그래서 각 CPU마다 각자 부속된 메모리를 나누어 줌.

-> 메모리 위치에 따라 접근하는 속도가 일정하지 않음.

CPU는 자기 바로 옆에 붙어 있는 가까운 메모리를 가장 먼저 사용.

CPU3 -> memory0으로 가는 것 가능! 다만 Access Time이 느리다.



## Clustered System 클러스터 시스템

멀티프로세서와 시스템과 비슷하지만, 여러 개의 개별 시스템이 함께 작동하는 방식.

High-Availability(고가용성) 서비스 제공

Asymmetric clustering 비대칭형 클러스터링 : 한 대의 기기가 Hot-Standby 모드로 대기

HA(High Availability) 클러스터 : Failover(주 서버가 고장나면 대기 서버가 그 일을 받아서 함)

Active-Standby 모드 : 서버 한대를 Active로, 다른 한대로 Standby

Symmetric clustering 대칭형 클러스터링 : 여러 노드가 동시에 어플리케이션을 실행하며, 서로 모니터링

LB(Load Balancing) 클러스터 : 작업을 여러 서버에 골고루 나누어 줌. 부하 분산.

Active-Active 모드 : 노는 서버 없이 모두 Active 상태로 일을 함. 모든 작업을 같이 처리.

일부 클러스터는 HPC(고성능 컴퓨팅)을 목적으로 구축함

병렬화(Parallelization) 아키텍처 필수

DLM(Distributed Lock Manager) : 작업 충돌 방지. 한 서버가 데이터를 쓰는 동안 다른 서버가 접근하지 못하도록 Lock을 해주는 역할.

SAN(Storage-Area Network)을 통해서 스토리지 공유. 하나의 큰 저장장치 만들어 놓고 여러 서버가 공유. 마치 자기 본체에 꽂힌 물리적 하드디스크처럼 사용.

## 멀티프로그래밍 Multiprogramming(Batch Sstsem)

\* Batch System : 유사한 성격의 작업(Job)들을 미리 모아두었다가 한꺼번에 순차적으로 처리

싱글프로그래밍 -> 그 프로그램이 I/O를 하는 동안 CPU는 놀고 있음(idle)

멀티프로그래밍 -> 한 프로그램이 쉴 때 CPU가 다른 프로그램으로 넘어가서 CPU가 계속 일함(busy)

### ★Process 프로세스 : 현재 실행중인 프로그램★

CPU Scheduling을 통해서 하나의 작업이 선택되어 실행됨.

ex) I/O 등을 위해 대기해야 할 때, OS가 다른 작업으로 전환함

## 멀티태스킹 Multitasking(시분할 Time-Sharing)

: 멀티프로그래밍의 논리적 확장. 대화형 컴퓨팅(CPU가 작업을 빈번하게 전환, 실행 중인 각 작업과 상호작용 可)

응답 기준 : 1초 미만! -> 기다린다는 느낌이 없게 함. 1:1로 쓰고 있고, 실시간으로 쓰고 있다고 느낌

CPU 스케줄링 : 여러 작업이 동시에 실행할 준비가 되어 있으면 CPU 스케줄링이 필요

가상 메모리 : 실제 메모리는 작더라도, HDD나 SSD같은 보조저장장치를 써서 "메모리 용량보다 많은 프로세스를 동시에 올릴 수 있게 함.

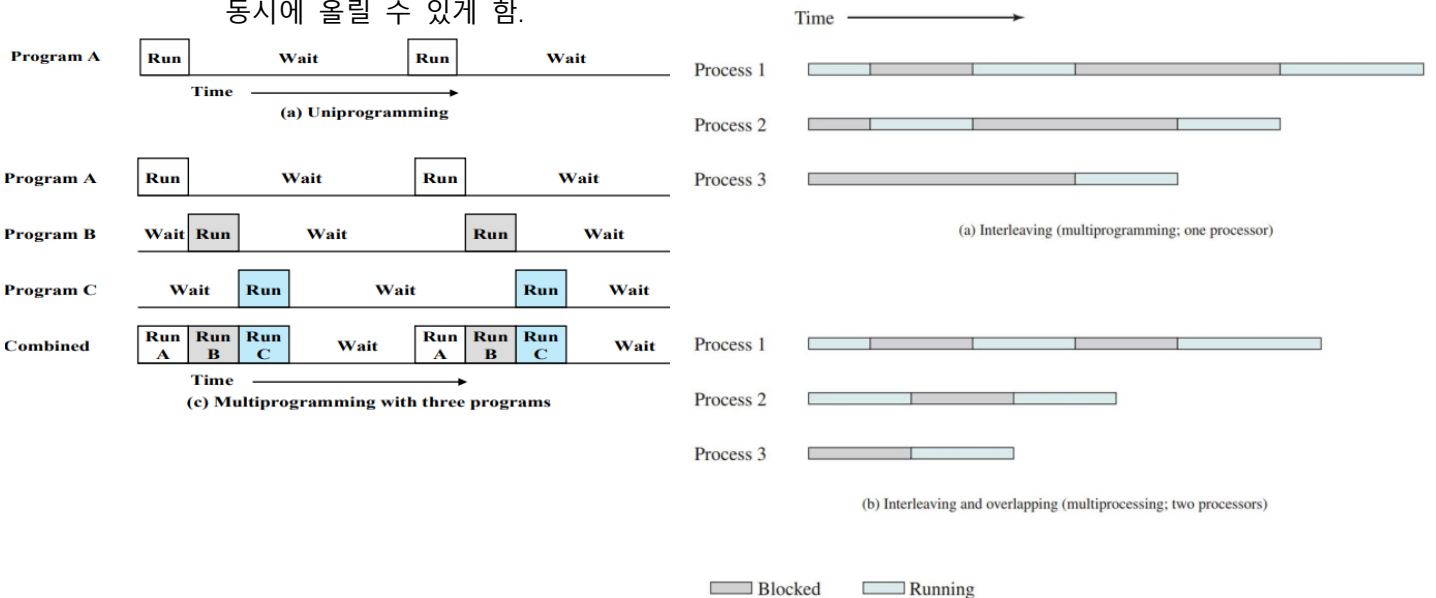


Figure 2.12 Multiprogramming and Multiprocessing

| 용어                       | 정의                         | 그림 속 특징        |
|--------------------------|----------------------------|----------------|
| 멀티프로그래밍 Multiprogramming | 메모리에 여러 프로그램을 올려서 실행하는 것   | (a), (b) 공통 사항 |
| 멀티프로세서 Multiprocessor    | CPU 하드웨어가 2개 이상 있다는 뜻      | (b) 환경         |
| 멀티프로세싱 Multiprocessing   | 여러 CPU가 한꺼번에 연산을 처리하는 것    | (b)의 동작 방식     |
| 인터리빙 Interleaving        | CPU 하나가 작업을 번갈아 가며 수행하는 것  | (a)의 동작 방식     |
| 오버래핑 Overlapping         | 여러 CPU가 물리적으로 동시에 여러 연산 처리 | (b)에서 하늘색 겹 부분 |

### 듀얼-모드 연산

듀얼 모드 : OS가 OS 자신과 다른 시스템 구성 요소를 보호.

유저 모드 vs 커널 모드 : 모드가 나누어져 있음.

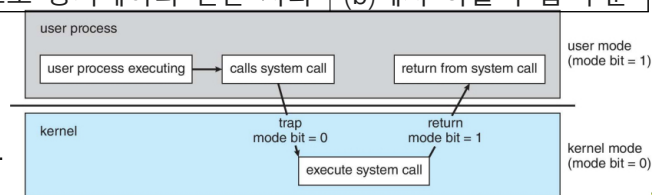
Mode bit 모드 비트 : 현재 시스템이 어떤 모드인지 나타내기 위해 제공되는 비트.

사용자가 user code를 실행중인지 kernel code인지 구분할 수 있게함

일부 명령어는 Privileged 특권 명령어로 지정되어 커널에서만 실행 가능

\* Privileged 특권 명령어 : 커널 명령어 중 시스템 전체에 영향을 주는 특정 동작. ex) I/O 제어, 타이머 System call이 모드를 kernel로 변경. 호출에서 return하면 다시 user 모드로 변경

kernel mode = 0      user mode = 1



## 타이머 Timer

: 무한 루프나 프로세스가 자원을 독점(hogging) 하는 것을 방지

일정 시간이 지난 후 컴퓨터에 **인터럽트**를 발생시키도록 타이머가 설정됨  
물리적인 시계(하드웨어 클락)에 의해서 값이 감소되는 카운터를 만들어 놓음  
카운터 값이 **0이 되면 인터럽트!**

OS가 카운터를 설정한다. **Privileged 특권 명령어**이다!(일반 사용자가 수정 불가)

## 프로세스 관리 Process Management

**프로세스** : 실행 중인 프로그램. 작업의 단위.

프로세스는 CPU, 메모리, I/O, 파일, 초기화 데이터와 같은 자원이 필요함.

프로세스가 종료되면, OS가 재사용 가능한 자원을 회수(reclaim)함.

프로그램 : Passive Entity 수동적인 개체

프로세스 : Active Entity 능동적인 개체

**싱글스레드 프로세스 Single-Thread Process** : 하나의 PC가 있음. 프로세스는 완료될 때까지 한 번에 하나씩 명령어를 순차적으로 실행.

**멀티스레드 프로세스 Multi-Thread Process** : 스레드 당 하나의 PC가 있음.

## 프로세스 관리 활동

- \* 사용자 및 시스템 프로세스의 **생성과 삭제**
- \* 프로세스의 **일시 중단과 재개**
- \* 프로세스 **동기화** Process Synchronization 제공
- \* 프로세스 **간 통신** Process Communication 제공
- \* **데드락 핸들링** Deadlock Handling 제공

## 메모리 관리 Memory Management

프로그램을 실행하려면 모든(또는 일부)**명령어가 메모리에 있어야 함**.

프로그램에 필요한 **데이터도 메모리에 있어야 함**.

**메모리 관리** : 무엇이 언제 메모리에 있을지 결정. CPU 사용률과 사용자 응답성을 최적화.

- \* 메모리의 어느 부분이 누구에 의해서 사용되는지 **추적 track** 함
- \* 어떤 프로세스와 데이터를 **메모리로 이동**시킬지 결정
- \* 메모리 공간 할당 Allocation 또는 해제 Deallocation

## 캐싱 Caching

데이터를 처리할 때마다 느린 디스크에서 가져오면 너무 오래 걸리므로, 자주 쓰는 데이터를 잠깐 "빠른 곳"으로 옮김

메인 메모리는 HDD/SSD의 cache로 간주될 수 있음. 일종의 캐싱 행위이다!

CPU는 데이터를 찾을 때 레지스터 -> 캐시 -> 메인 메모리 순서로 **빠른 곳부터 확인**  
**있으면 Cache hit, 없으면 Cache miss**이고, 데이터를 캐시로 복사해서 사용.

캐시는 크기가 제한적임 -> 캐시 관리가 중요함!

| Level                     | 1                                      | 2                             | 3                | 4                | 5                |
|---------------------------|----------------------------------------|-------------------------------|------------------|------------------|------------------|
| Name                      | registers                              | cache                         | main memory      | solid-state disk | magnetic disk    |
| Typical size              | < 1 KB                                 | < 16MB                        | < 64GB           | < 1 TB           | < 10 TB          |
| Implementation technology | custom memory with multiple ports CMOS | on-chip or off-chip CMOS SRAM | CMOS SRAM        | flash memory     | magnetic disk    |
| Access time (ns)          | 0.25-0.5                               | 0.5-25                        | 80-250           | 25,000-50,000    | 5,000,000        |
| Bandwidth (MB/sec)        | 20,000-100,000                         | 5,000-10,000                  | 1,000-5,000      | 500              | 20-150           |
| Managed by                | compiler                               | hardware                      | operating system | operating system | operating system |
| Backed by                 | cache                                  | main memory                   | disk             | disk             | disk or tape     |

Movement between levels of storage hierarchy can be explicit or implicit

HW가 관리 (CPU)

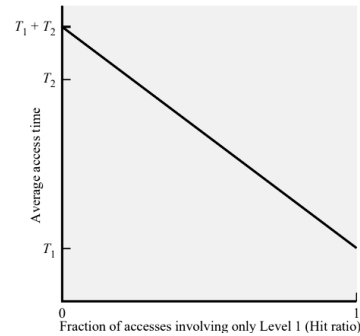
X축 : 레벨 1(더 빠른 메모리)에서만 이루어지는 히트율

Y축 : 평균 접근 시간

$T_1$  : 히트율이 100프로. 모든 데이터가 레벨 1 메모리에 있음

$T_1+T_2$  : 히트율이 0프로. 하나도 없어서 레벨 2메모리에서 가져옴

가정:  $T_1$  접근 시간 =  $0.1ms$ ,  $T_2$  접근 시간 =  $1ms$ , 히트율 = 95%



- $T_1$  : Access time to level 1
- $T_2$  : Access time to level 2

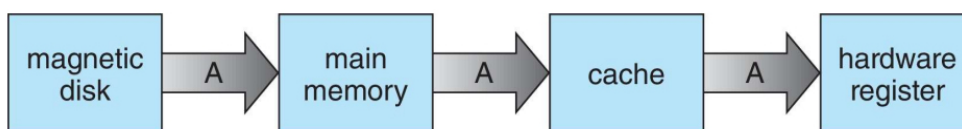
$$\text{Average Access Time} = (0.95 \times 0.1) + 0.05 \times (0.1 + 1)$$

$$\Rightarrow 0.095 + 0.05 \times 1.1 = 0.095 + 0.055 = 0.15ms$$

- 결과: 히트율이 95%만 되어도, 평균 속도는 느린 장치( $1ms$ )가 아니라 빠른 장치( $0.1ms$ )에 매우 가까운

\*\*0.15ms\*\*가 됩니다. ☺

디스크에서 레지스터로 데이터의 이동



멀티태스킹 환경에서는 데이터가 어디에 저장되어 있는 가장 최신 값을 사용해야 함. 바뀐 값을 쓰면 안됨!

캐시 일관성(cache coherency) : 모든 CPU가 자신의 캐시에 최신 값을 가질 수 있도록 해야 함.

분산 환경 Distributed environment에서는 훨씬 잡하고, 하나의 데이터에 대해 여러 개의 복사본이 있을 수 있음.

파일 시스템 관리

파일 : 물리적인 특성을 추상화하여 논리적 저장 단위로 만든 것. 보통 디렉토리로 조직화됨

매체 Medium : 디바이스(i.e. 디스크 드라이브)에 의해 제어됨.

접근 제어 Access Control : 누가 무엇에 접근 할 수 있는가?

OS의 활동

- \* 파일 및 디렉토리 생성/삭제
- \* 파일과 디렉토리를 조작하기 위한 기본 기능(primitives) 제공 ex) open, read, write
- \* 파일을 보조 저장 장치에 매핑. 논리적인 파일 데이터가 실제로 하드디스크의 어디에 저장될지 연결.
- \* 파일을 안정적인 비휘발성 non-volatile 매체에 백업.

## 대량 스토리지 관리

메인 메모리에 담기에 너무 크거나 오랫동안 보관해야 하는 데이터를 저장하기 위해 **디스크 사용**  
어떤 스토리지는 굳이 빠를 필요 없음.

3차 저장장치 Tertiary storage : 광학 저장 장치(CD/DVD), 자기 테이프

## OS의 활동

- \* 마운팅/디마운팅
- \* 빈 공간 관리
- \* 저장 공간 할당 : 파일에 디스크의 실제 위치를 나누어 주는 일
- \* 디스크 스케줄링 : 어디서부터 읽으면 빠를까?
- \* 파티셔닝 : 하나의 물리적 디스크를 여러 개의 논리적 구역으로 나누어 관리
- \* 보호 : 허가되지 않은 사용자로부터 접근 제한

## 보호와 보안

**보호 Protection** : OS에 의해 프로세스나 사용자의 접근을 제어하기 위한 매커니즘

**보안 Security** : 내부 및 외부 공격으로부터 시스템을 방어하는 것. by DoS, 웜, 바이러스, 신분 도용, 탈취 등...

**User identities UID 사용자 식별자** : 사용자 별로 부여. 접근 제어를 결정

**Group identifier GID 그룹 식별자** : 시스템 관리자로 등록하면 어디까지 접근하고, 일반인은 접근 못하게 함

## 가상화

**다양한 OS 사용** : 각 VM에서 다른 OS 사용 가능

**안정성 향상** : 서버 고장을 대비해 가상 서버를 수십개 만들어 놓음

**자원의 효율성** : 실제 물리 서버는 하나지만, 가상화를 통해 마치 여러 대의 서버가 있는 것처럼 나누어씀.

하드웨어 활용도를 극대화

## 분산 시스템 Distributed System

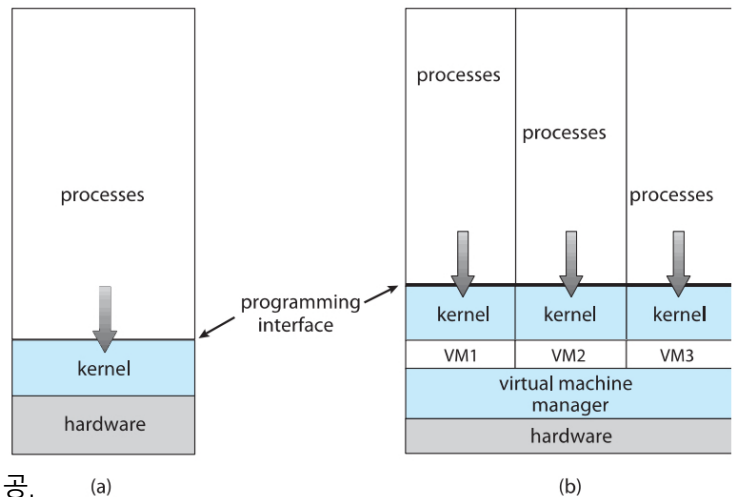
별개의 이질적인 시스템의 집합.

여러 컴퓨터를 네트워크로 묶여 있음.

**네트워크** : 통신 경로. TCP/IP가 일반적

LAN : Local Area Network

WAN : Wide Area Network



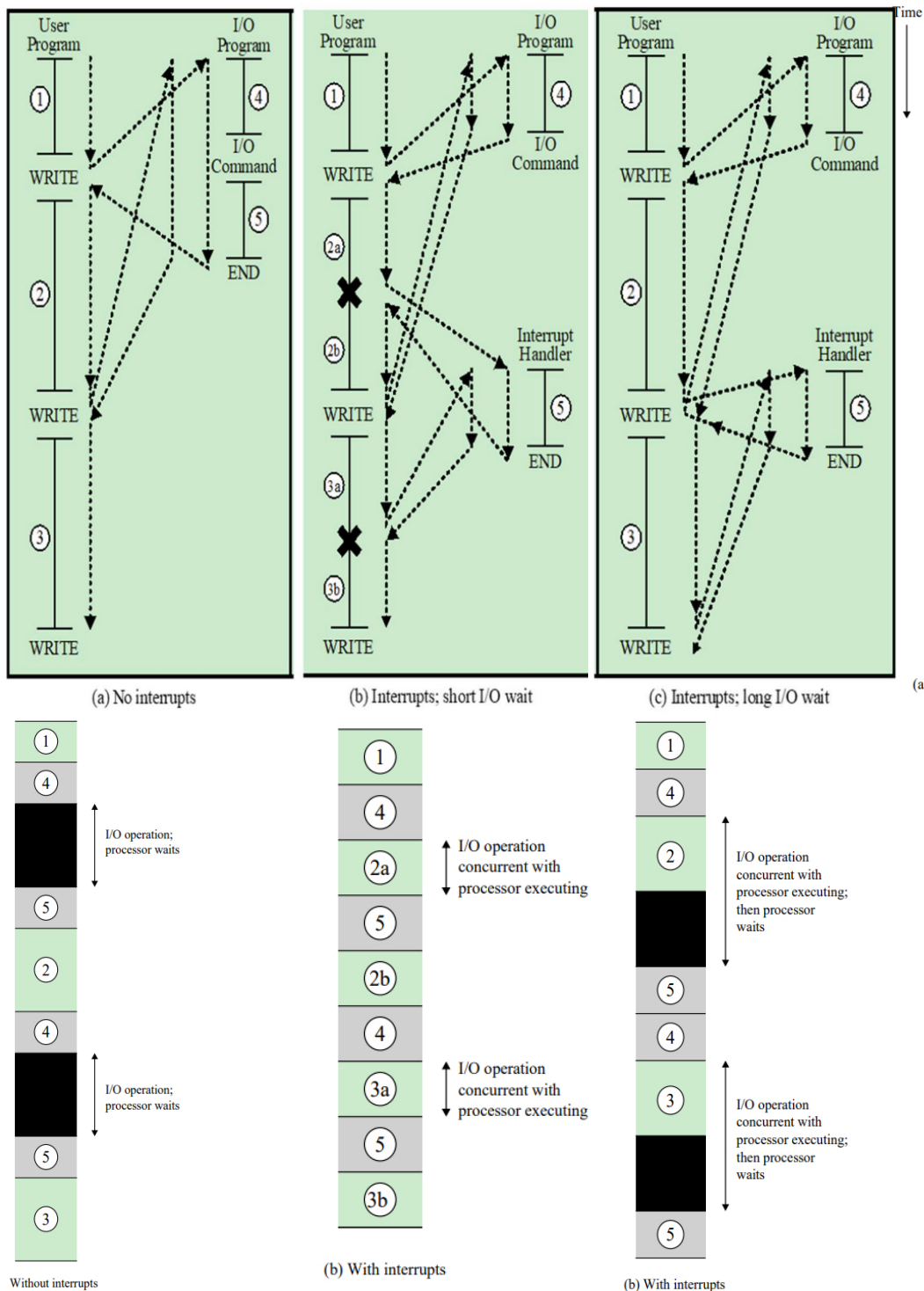
**네트워크 OS**는 네트워크를 통해 시스템 간 기능을 제공. (a)

(b)

\* 분산 시스템이 서로 데이터를 주고받으며 협업할 수 있도록 하는 규칙

\* 사용자는 마치 성능이 아주 좋은 단 한 대의 컴퓨터를 쓰고 있는 것처럼 느낌. 실제로는 수백 대의 컴퓨터.

## Interrupt 인터럽트



## 3주차 Process 프로세스

OS는 프로그램을 “프로세스 단위”로 관리하며 이를 구동함.

### 프로세스 Process

: 실행 중인 프로그램. 프로세스의 Instruction은 sequential하게 순차적으로 수행됨.

**프로그램** : Passive 수동적인 개체.

**프로세스** : Active 능동적인 개체

실행 파일이 메모리에 올라가서 Active한 상태가 되어야 비로소 프로세스라고 부를 수 있다.

★ 한 개의 프로그램은 여러 개의 프로세스가 될 수 있다 e.g. 크롬에서 창을 여러 개 열었을 때  
프로세스는 자원이 필요. CPU, 메모리, I/O 장치, 파일 등...



프로세스가 메모리에 로드되었을 때

Stack 스택

함수 매개변수, 리턴 주소, 지역변수  
OS가 관리함. 함수 호출 시 자동생성, 사라짐

함수 호출 시 임시 데이터 저장소

Heap 힙

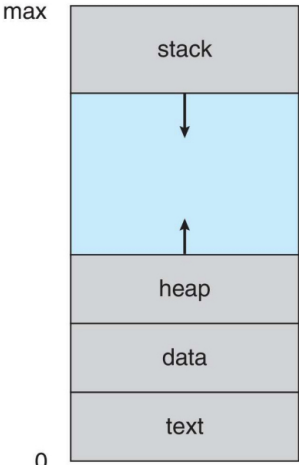
런타임에 동적할당된 메모리  
프로그래머가 직접 관리해야 함! 메모리 해제 등

Data 데이터

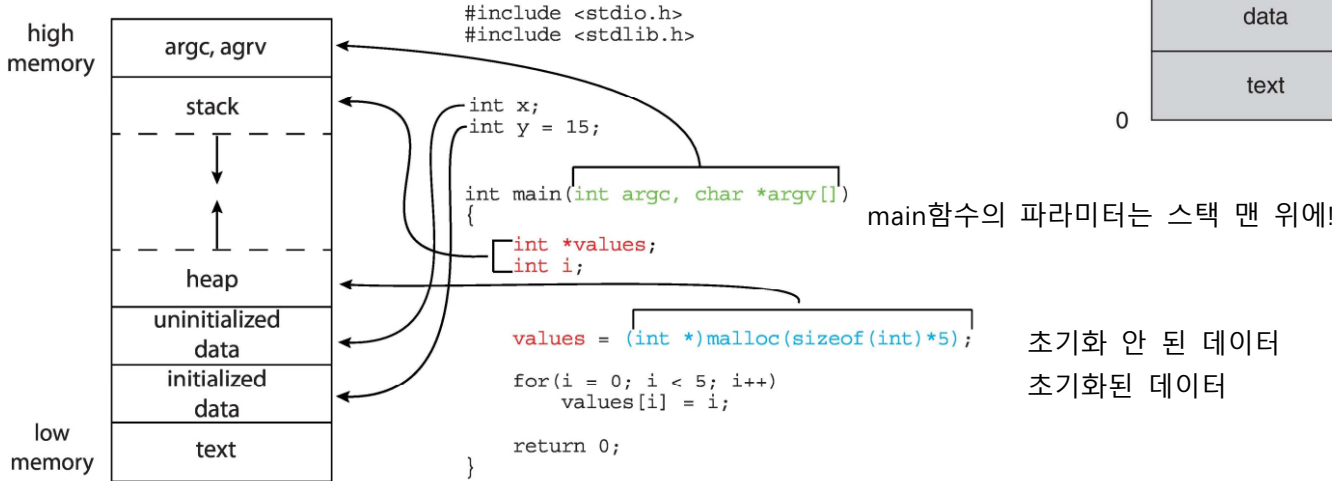
전역변수

Text 텍스트

프로그램 코드



32비트/64비트 컴퓨터에 따라서 논리적 주소 체계에 따라 결정됨.



프로세스 상태

Trace 트레이스 : 이러한 일련의 instruction들을 줄 세워서 출력할 수 있게끔 한 것

CPU가 몇 번지 명령어를 실행했는지 그 주소들을 순서대로 쭉 적어놓은 리스트

Dispatcher 디스패처 : 현재 실행 중인 프로세스를 다른 프로세스로 바꾸는 작업을 하는 작은 프로그램  
kernel에서 작동. 프로세스라고 부르지 않음.

100-105는 프로세스 교체하는 코드. by Dispatcher

6개의 Instruction이 실행되면 Timeout -> 프로세스 교체!  
(Dispatcher가)

도중 I/O Request가 일어나도 프로세스 교체

색칠된 부분은 실제 일을 한 것이 아니고 작업 준비 과정.  
색칠한 부분이 많아지면 오버헤드로 작용할 수 있음.

|                   |       |               |       |
|-------------------|-------|---------------|-------|
| 1                 | 5000  | 27            | 12004 |
| 2                 | 5001  | 28            | 12005 |
| 3                 | 5002  | ----- Timeout |       |
| 4                 | 5003  | 29            | 100   |
| 5                 | 5004  | 30            | 101   |
| 6                 | 5005  | 31            | 102   |
| ----- Timeout     |       | 32            | 103   |
| 7                 | 100   | 33            | 104   |
| 8                 | 101   | 34            | 105   |
| 9                 | 102   | 35            | 5006  |
| 10                | 103   | 36            | 5007  |
| 11                | 104   | 37            | 5008  |
| 12                | 105   | 38            | 5009  |
| 13                | 8000  | 39            | 5010  |
| 14                | 8001  | 40            | 5011  |
| 15                | 8002  | ----- Timeout |       |
| 16                | 8003  | 41            | 100   |
| ----- I/O Request |       | 42            | 101   |
| 17                | 100   | 43            | 102   |
| 18                | 101   | 44            | 103   |
| 19                | 102   | 45            | 104   |
| 20                | 103   | 46            | 105   |
| 21                | 104   | 47            | 12006 |
| 22                | 105   | 48            | 12007 |
| 23                | 12000 | 49            | 12008 |
| 24                | 12001 | 50            | 12009 |
| 25                | 12002 | 51            | 12010 |
| 26                | 12003 | 52            | 12011 |
|                   |       | ----- Timeout |       |

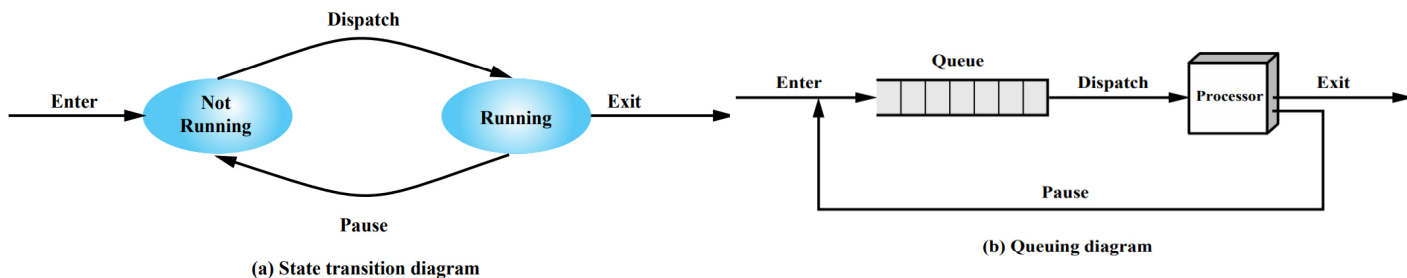


## Two-State Process Model

## 2상태 프로세스 모델

CPU가 작업을 수행하는 중인지 아닌지에 따라 상태를 아주 단순하게 구분 가능.

but, I/O를 기다리는 프로세스와 당장 CPU만 있으면 실행할 수 있는 프로세스가 똑같이 Not Running이라는 하나의 Queue에 섞여 있게 됨.



## Five-State Process Model

## 5상태 프로세스 모델

2상태에 비해서 Ready와 block을 따로 관리하므로, CPU는 준비 큐에서만 프로세스를 꺼내오면 되어 시스템이 훨씬 빠르게 움직일 수 있음.

**New :** 프로세스가 Passive에서 Active 상태로 바뀌는 도중에 있는 단계.

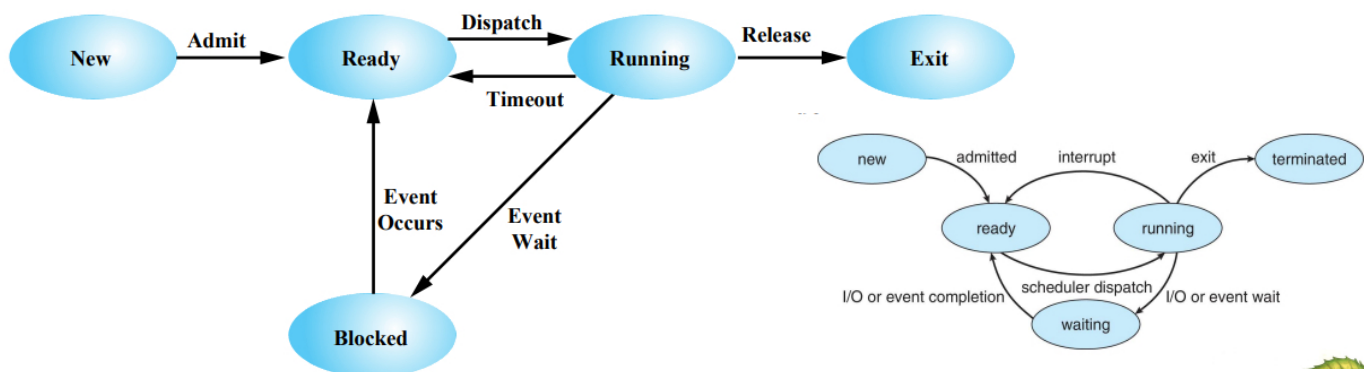
메모리에 올리고 있는 과정. 아직 올라온 건 아님. 과도기적 과정

**Running :** 실제로 CPU에서 명령어가 실행되는 중

**Blocked(Waiting) :** 이벤트를 기다리고 있는 상태. e.g. I/O 요청을 하고 결과가 올 때 까지 기다리는 프로세스

**Ready :** 스케줄링이 되면 바로 Running으로 갈 수 있는 상태. 메모리에 올라가서 대기하고 있는 상태.

**Exit(Terminated) :** 프로세스가 실행을 마친 상태



**admit :** new 상태에서 생성이 완료되어 ready 큐에 들어가는 과정

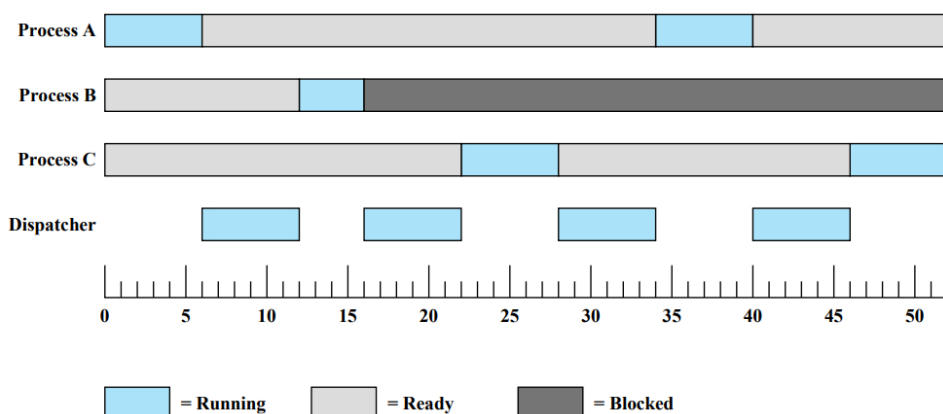
**dispatch :** ready 상태인 프로세스를 하나 골라 CPU에 올려 실행시킴. Dispatcher가 담당함

**timeout/interrupt :** 실행중이던 프로세스가 다시 ready 상태로 변함

**Event Wait :** 실행 중 입출력이 필요하면 CPU를 반납하고 waiting 상태로 내려가 이벤트를 기다림

**Event Occurs/Completion :** 기다리던 입출력이 끝나면 다시 실행될 수 있도록 ready 상태로 올라감

**exit :** 모든 작업을 마친 프로세스가 종료되는 경로



PCB Process Control Block

: 각 프로세스는 운영체제 내에서 PCB에 의해 표현됨. Task Control Block이라고도 부름.  
프로세스의 모든 상태 정보가 들어 있는 곳. OS가 프로세스를 관리하기 위해 사용하는 핵심 블록

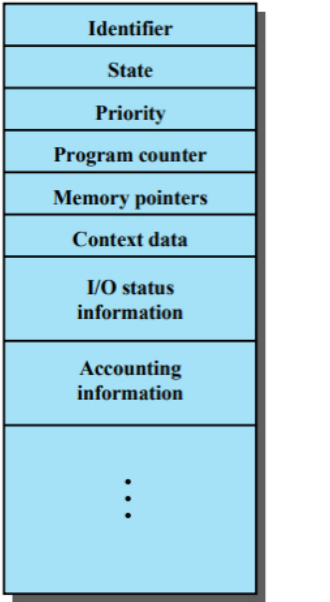
PID : Process ID. 프로세스마다 부여되는 고유 번호. 중복 X DB의 Primary Key, 주민등록번호...

Process State Information 프로세스 상태 정보

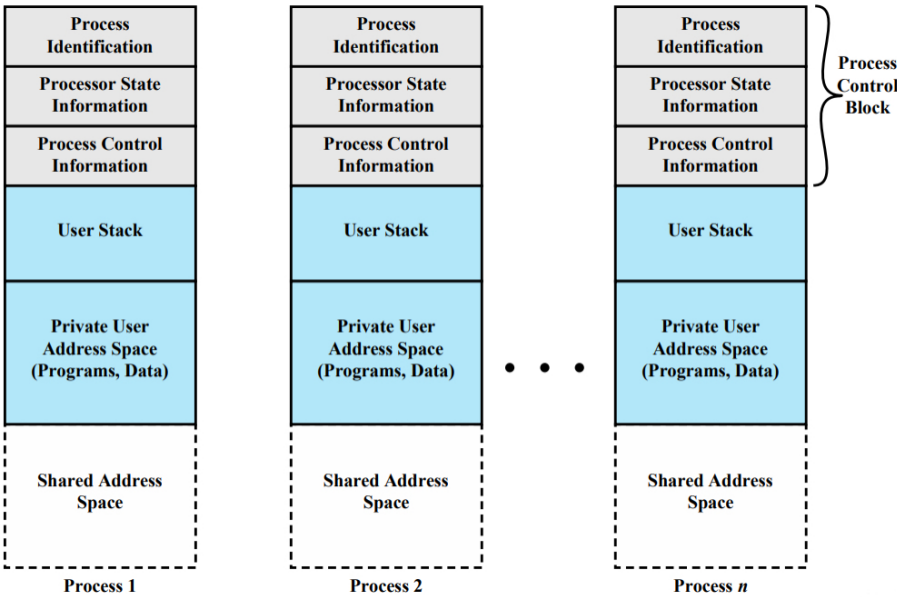
Program Counter (PC) : 다음에 실행할 명령어의 위치  
Context Data(CPU Register) : CPU 내부 레지스터에 담겨 있던 프로세스의 모든 정보

Process Control Information 프로세스 제어 정보

Process State 프로세스 상태 : Running, Waiting etc...  
CPU 스케줄링 정보 : Priority 어떤 프로세스를 먼저 처리할까?  
메모리 관리 정보 : 프로세스가 메모리의 어느 위치에 올라와 있는가?  
계정 정보 : CPU 사용량, 시작부터의 clock 시간, Timeout 등 관리  
I/O 상태 정보 : 현재 할당된 I/O 장치나 열려 있는 파일 목록을 관리함.



3.1 Simplified Process Control Block



프로세스의 맨 위에는 PCB가 있음. PID, 프로세스 상태정보, 프로세스 제어정보  
중간 - User Stack(위 -> 아래) + Private User Address Space(아래 -> 위)

실제로는 PCB는 커널영역에, 프로그램 실행 데이터(Stack, Heap 등)은 유저 영역에 존재.  
=> 하지만 논리적으로는 하나로 묶어서 “프로세스 이미지”라고 부름

프로세스 스케줄링

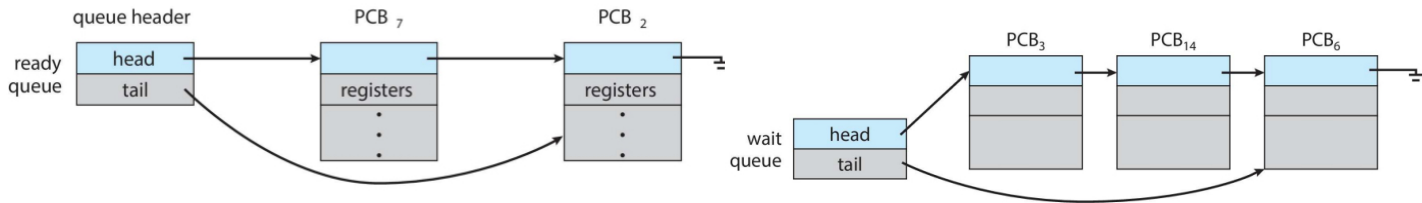
CPU 사용 최대화를 위함.

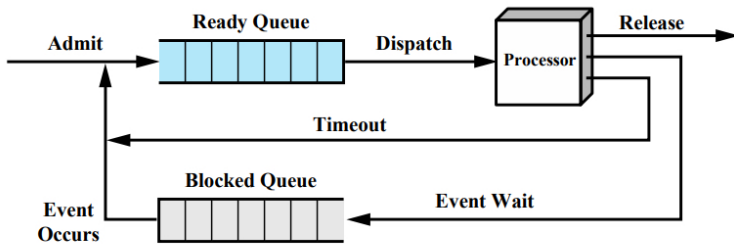
CPU 스케줄러 : CPU에서 다음에 실행시킬 지 선택. 프로세스들의 스케줄링 큐를 유지 관리함.

Ready Queue : 메인 메모리에 상주하며 Ready 상태인 프로세스가 대기 중인 큐

Blocked Queue : I/O 요청 등을 하고 그 결과가 올 때까지 대기하는큐

단일 Blocked 큐





(a) Single blocked queue

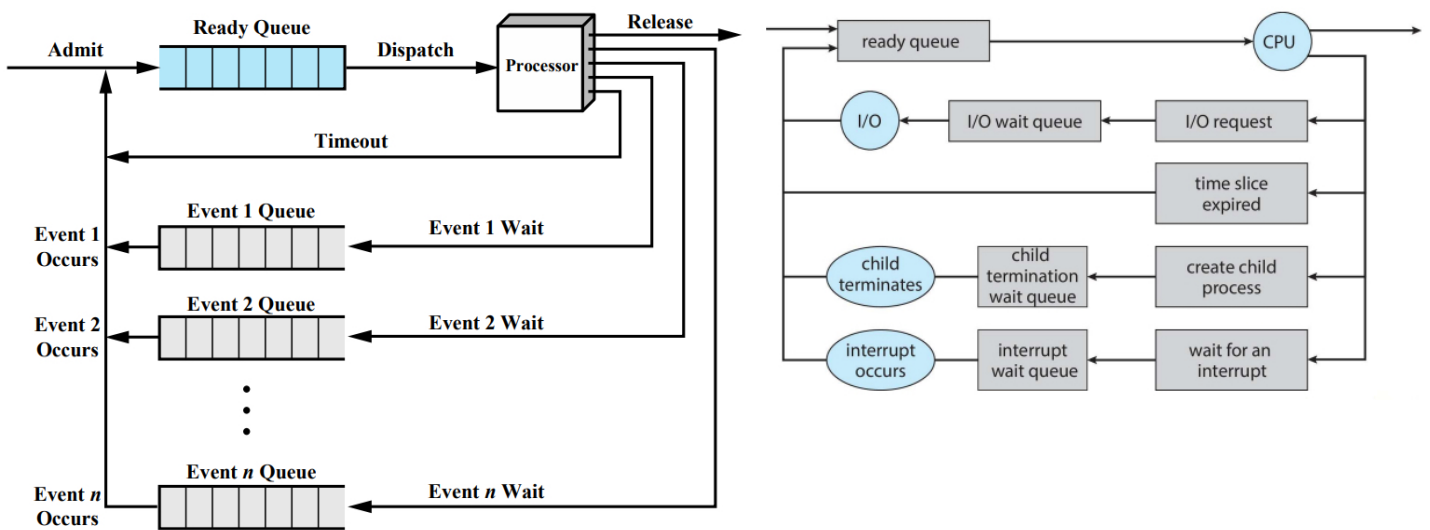
이 방식에서 큐는 linked list 구조로 구현되어 있다!

이 방식에서는 하나의 Blocked 큐에 모든 대기 프로세스가 섞여 있다. 그런데, 특정 이벤트가 끝났을 때 이 큐를 전체 탐색을 해서 찾아야 하기 때문에 Search time이 발생한다.

## 다중 Blocked 큐

Blocked에 있는 프로세스를 이벤트 종류에 따라서 관리

특정 이벤트가 해결되면, 해당 Event 큐에 대기하던 프로세스들을 한꺼번에 Ready 큐로 옮길 수 있어 매우 빠르고 효율적 => 퍼포먼스 높아짐!



(b) Multiple blocked queues

## Swapping

: 프로세스의 일부 또는 전부를 메인 메모리에서 디스크로 이동시키는 것

메모리에서 idle인 프로세스가 있으면, OS는 공간을 확보하기 위해 디스크로 보냄.

메인 메모리에 있는 어떤 프로세스도 Ready 상태가 아닐 때, OS는 Blocked 프로세스 중 하나를 디스크의 Suspend Queue로 스왑 아웃. 그런 다음 다른 프로세스를 가져옴.

Suspend Queue 지연 큐 : 일시적으로 쫓겨나가거나 지연된 기존 프로세스들의 큐.

Swapping을 하면 이거 자체가 I/O 작업이기 때문에 문제를 더 악화시킬 수도 있음.

but, 디스크 I/O는 다른 I/O에 비해서는 빠르기 때문에 보통 성능을 향상시킴.

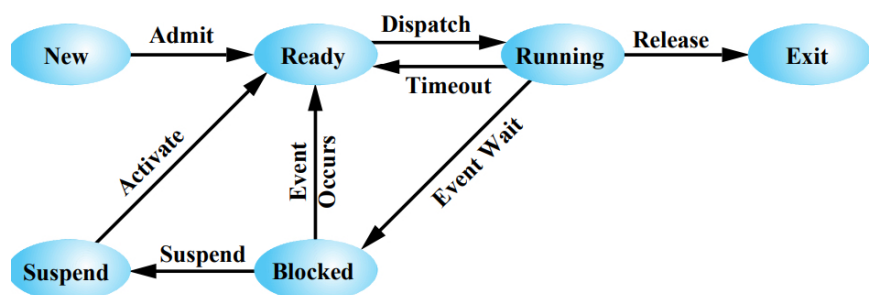
## Suspend-State Process Model

- With One Suspend State

suspend : 메모리에 있는 모든 프로세스가

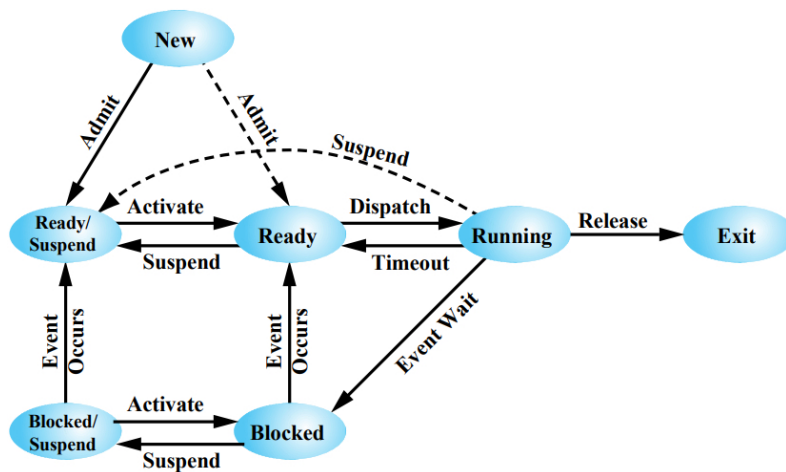
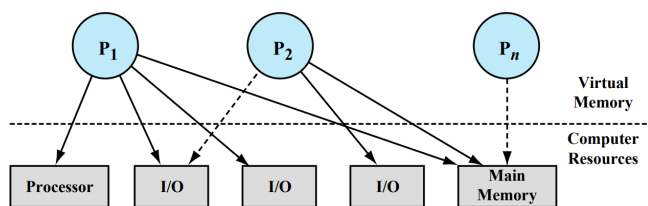
block 상태여서 suspend로 일부 보냄

active : 메모리 여유 생기면 다시 메모리로



(a) With One Suspend State

- With Two Suspend State



(b) With Two Suspend States

**New** : 프로세스가 막 실행되어 Ready가 되기 전 단계. Passive -> Active로 바뀌는 도중

**Ready/Suspend** : 디스크에 있지만, 메모리에 올리기만 하면 실행 가능

**Blocked/Suspend** : 디스크에 있으면서 I/O같은 이벤트가 끝나기를 기다리는 상태

**Ready** : 프로세스가 생성되어 메모리에 올라가 있는 상태. CPU만 할당받으면 즉시 실행될 수 있는 상태

**Running** : 실제로 CPU를 점유하여 명령어를 수행하고 있는 상태.

**Blocked** : I/O 요청과 같이 특정 이벤트가 끝나기를 기다리는 상태

**Exit** : 프로세스가 모든 실행을 마친 상태.

**Suspend(Running -> Suspend/Ready)** : Timeout이 걸렸는데 우선순위에 밀려서 메모리에 있을 공간이 없을 때.

e.g. 불이 난 경우. 소방관이 Ready에 들어와야 하기 때문에 우선순위를 소방관에 줘야해서 원래 있던 프로세스는 뒤로 빠져야 함. 우선순위가 낮으면 Ready로 가지 못하고 Ready/Suspend로 가게 됨.

**Admit(New -> Ready)** : 프로세스가 생성되었을 때, 메모리에 수용할 공간이 충분할 때

**Admit(New -> Ready/Suspend)** : 프로세스가 생성되었지만, 메모리가 꽉 차있어서 들어갈 수 없을 때.

P<sub>1</sub> : **Running** 상태. Processor를 장악.

P<sub>2</sub> : Processor 보유X, 메모리에 올리와 있음. I/O와 연결되어 있으니 **Blocked** 상태

P<sub>n</sub> : 메인 메모리를 가지고 있지 않음(점선!) Swap되어 디스크에 있음. **Suspend** 상태

## OS 제어 테이블의 일반적 구조

**메모리 테이블** : 프로세스에 할당된 메인메모리, 보조저장장치 정보, 가상메모리 정보 등

**I/O 테이블** : I/O 장치, 채널 관리하기 위해 사용. 프로세스 할당된 장치 정보

**파일 테이블** : 파일의 존재여부, 보조 기억장치 내 위치, 현재 상태, 기타 정보

**Primary 프로세스 테이블** : 프로세스를 총괄적으로 관리하는 가장 중요한 테이블

목록과 프로세스의 기본 정보를 담은 테이블. **PID로 구성.**

커널 영역에서 관리!

## Context Switch

인터럽트가 발생하면 시스템은 CPU에서 실행중인

프로세스의 현재 Context를 저장해야 함.

처리가 끝나면 해당 Context를 복원 할 수 없음

Context(PC, 레지스터 값)는 PCB에 저장이 된다.

**Context Switch** : CPU 코어를 다른 프로그램으로 바꾸는 것.

현재 상태를 저장하고, 다른 프로세스를 복원한다,

Context Switch를 하는 시간은 **오버헤드**이다! 유용한 일을 하지 않음.

OS복잡성, PCB 복잡성  $\propto$  Context Switch 시간

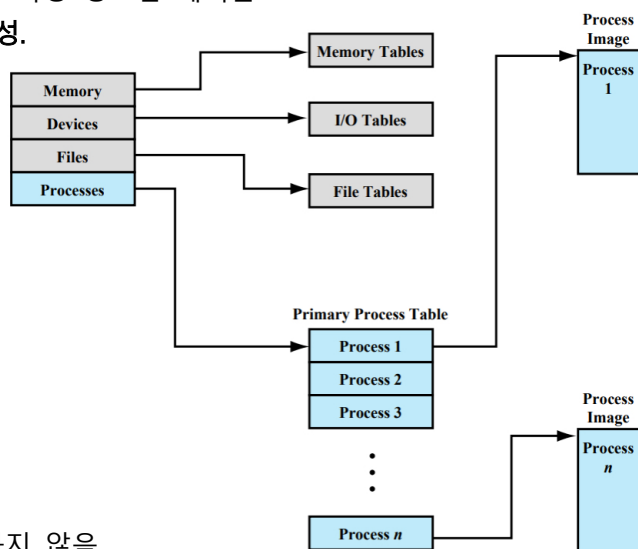
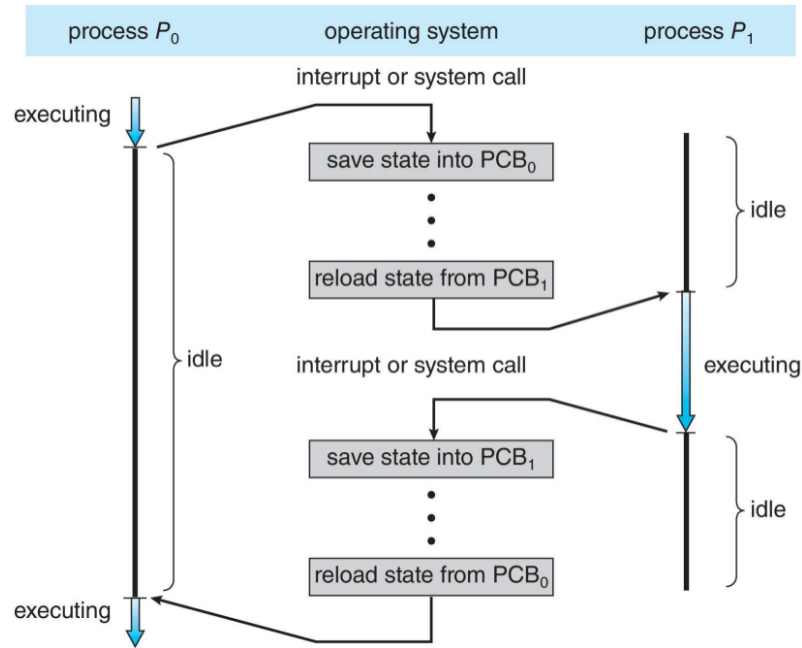


Figure 3.11 General Structure of Operating System Control Table

일부 하드웨어는 CPU당 여러 세트의 레지스터를 제공함.  
 한 번에 여러 Context Switch 할 수 있음. -> 하지만 여전히 한계 有



### 프로세스에서의 연산

OS는 프로세스 생성/종료에 관한 메커니즘을 제공해야 함.

**새로운 Batch job(새로운 일괄 처리 작업 ≈ 프로그램 시작) :** 시스템이 미리 정해진 작업을 수행하기 위해 새로운 프로세스를 메모리에 올리는 단계.

**Interactive logon(대화형 로그인) :** 사용자가 터미널을 통해 시스템에 접근하여 로그인을 하면, 유저를 위한 프로세스가 하나 생성됨.

**Created by OS to provide a service :** 사용자가 기다릴 필요 없이 user program을 대신해 기능을 수행할 프로세스를 생성할 수 있음(e.g. 프린팅 제어 프로세스)

**Spawned by existing process :** 모듈화나 병렬성 활용을 위해 사용자 프로그램이 여러 프로세스의 생성을 지시할 수 있음. -> Spawning : 실행 중인 프로세스가 다른 프로세스를 만드는 것.

### 프로세스 생성

**프로세스 Spawning :** 실행 중인 프로그램이 OS에 요청하여 새로운 프로세스를 만드는 연산

**Parent 부모 프로세스 :** 원래 존재하던, 새로운 프로세스를 만드는 프로세스

**Child 자식 프로세스 :** 새롭게 만들어진 프로세스. 부모로부터 리소스를 물려받거나 독립적으로 동작할 수 있음.

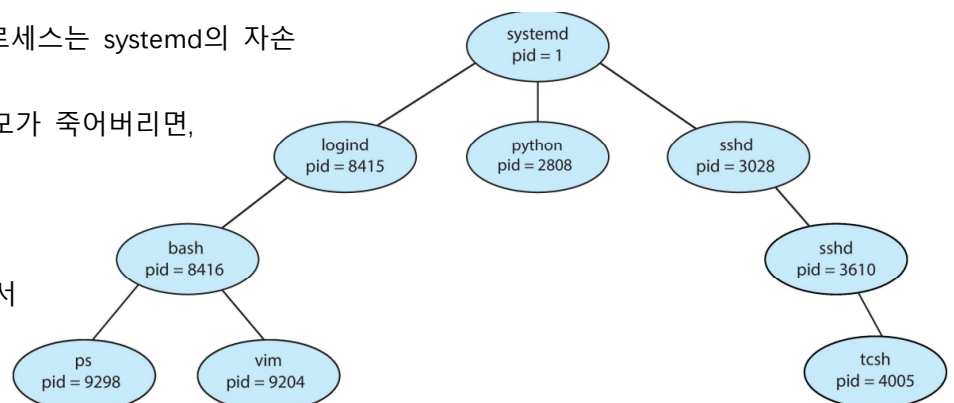
**PID : 나의 ID      PPID : 부모 프로세스의 PID**

Parent가 Child를 생성하고, 이 child가 다시 다른 프로세스를 생성하며 Tree를 형성함.

systemd : 최상위 Process. 모든 프로세스는 systemd의 자손

리눅스에서 자식을 챙기지 않고, 부모가 죽어버리면, 자식은 고아(Orphan)상태가 됨.  
 => 그러면 PPID 1로 만듦.

sysyemd는 주기적으로 wait()호출해서 고아/좀비 문제 해결!  
 (프로세스 테이블에서 완전히 삭제)



**자원 공유 옵션** : 부모 자식 간 리소스를 어떻게 나눌까?

방법 1) 부모가 가진 모든 리소스를 자식과 함께 사용

방법 2) 부모가 가진 리소스의 일부분만 공유

방법 3) 부모와 자식이 서로 독립적으로 리소스를 사용

**실행 옵션** : 어떤 순서나 방식으로 실행될까?

방법 1) 부모와 자식이 Concurrent하게 실행

방법 2) 부모가 자식이 종료될 때까지 기다림

**주소 공간 Address space**

자식 : 부모의 복제본

자식 프로세스에 새로운 프로그램이 로드됨.

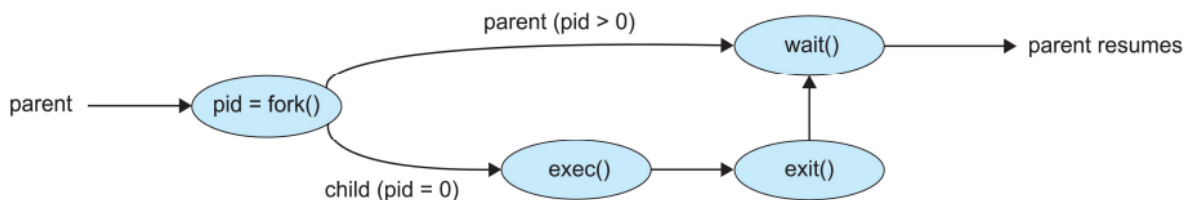
**UNIX 예시**

**fork() 시스템 콜** : 새로운 프로세스 생성

부모와 똑같은 복제된 자식 탄생

**exec() 시스템 콜** : fork()이후에 프로세스의 메모리 공간을 새로운 프로그램으로 교체하기 위해 사용

**wait()** : 호출하여 부모 프로세스는 자식이 종료될 때까지 기다림



fork()가 성공하면 -> 부모 프로세스는 방금 태어난 자식의 PID를 리턴받음

자식 프로세스는 0을 리턴받음

fork()가 실패하면 -> 음수값을 리턴

### 코드 1

```
int main() // main()함수는 메모리에서 Stack으로 코드 전체는 Text로
{
    pid_t pid; //지역변수는 Stack로

    /* fork a child process */
    pid = fork(); //fork()로 복제!

    if (pid < 0) { /* error occurred */ //오류 발생
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/lis", "lis", NULL); //exec() 명령으로 lis를 실행하는 코드로 덮어쓰기를 함
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL); //자식이 끝날 때까지 기다린다! 부모가 Blocked 상태
        printf("Child Complete");
    }

    return 0;
}
```

★만약★ fork() 바로 다음에 부모에 대해서 Timeout이 걸리면, else문 안에 wait()을 호출하지 못하고 Ready Queue 맨 뒤로 밀리고, 자식이 먼저 CPU를 차지하게 될 수도 있다.

## 코드 2

```
int value = 5;           // 전역 변수 선언: 데이터(data) 영역에 저장됨

int main()
{
    pid_t pid;

    pid = fork();    //복제!

    if (pid == 0) {        //자식 프로세스(child process)
        value += 15;        // 자식만의 독립된 메모리 공간에서 value 수정
        return 0;
    }
    else if (pid > 0) {    //부모 프로세스(parent process)
        wait(NULL);        //자식 프로세스가 종료될 때까지 대기 (Block 상태)
        printf("PARENT: value = %d", value);    //부모의 value는 수정되지 않고 그대로 5를 출력함

        return 0;
    }
}
```

여기서는 자식에서 value+=15를 해 줬지만, 부모는 완전 별개이므로 출력이 5가 나온다!

## 코드 3

```
int main() {
    fork();    // 이 시점에서 프로세스는 2개가 됩니다.
    fork();    // 기존 2개의 프로세스가 각각 자식을 낳아 총 4개가 됩니다.
    fork();    // 최종적으로 4개의 프로세스가 다시 복제되어 총 8개가 됩니다.

    return 0;
}
```

## 프로세스 종료

**exit()** : 프로세스가 마지막 문장을 실행한 후, **exit()** 시스템 콜을 사용하여 운영체제에 자신을 삭제해 달라고 요청  
모든 연산을 마쳤을 때 호출하는 방식. 프로세스가 가졌던 모든 자원을 운영체제에 return.

**wait()** : 자식에서 부모로 status data를 return함.

자식 프로세스가 종료되면서 자신의 종료 상태나 PID 정보를 부모에게 전달.

**abort()** : 부모가 **abort()** 시스템 콜을 하면 자식 프로세스의 실행을 종료시킬 수 있다.

Cascading termination 연쇄적 종료를 한다

모든 자식, 손자 등등이 종료됨.

종료는 OS에 의해 시작됨

프로세스가 종료되면 OS는 그 자원을 deallocate한다

**좀비 상태 Zombie** : 자식이 종료되었지만 부모가 아직 **wait()**을 통해 확인하지 않아 프로세스 테이블에 PID만 남아있는 상태.  
-> 모든 걸 반납하고 내 이름만 남아있는 상태      자식종료 -> **wait()** 전 짧은 상태

**고아 상태 Orphan** : 부모가 **wait()**을 호출하지 않고 먼저 종료된 경우.

리눅스에서는 **systemd**가 입양하여 주기적으로 정리해 줌.



## 4주차 Chapter 3: Process #2

### 멀티프로세스 아키텍처 : 크롬

예전에는 브라우저가 하나의 프로세스로 동작했음 but, 하나가 느려지면 전체 브라우저가 느려져서 못 쓰게 됨  
구글 크롬 브라우저는 3가지 다른 타입의 프로세스가 협력함.

1. **Browser** : UI, 디스크, 네트워크 I/O 관리                      브라우저 전체에서 하나 동작
2. **Renderer** : **웹페이지** 렌더링, **HTML** 다룸  
    **Sandbox** : 가둬놓고 돌림. 외부의 JS 코드가 시스템에 영향을 주지 못하도록 샌드박스에서 동작. 보안 취약점 최소화
3. **Plug-in** : 각 유형의 **플러그인(flash등)** 프로세스 생성.

### Interprocess Communication 프로세스 간 통신

OS에서 많은 프로그램이 도는데 독립적 Independent이거나 협력적Cooperating임.

#### Independent process 독립 프로세스

다른 프로세스에게 영향을 주거나 영향을 받지 않음  
다른 프로세스와 데이터를 공유하지 않음

#### Cooperating process 협력 프로세스

시스템에 실행되는 다른 프로세스에 영향을 주거나 받음.  
다른 프로세스와 데이터를 공유하는 프로세스는 협력 프로세스이다.

### Cooperating Process 협력 프로세스

**장점** : 정보 공유, 계산 속도 ↑, 모듈성 modularity(전체를 하나로 만드는 게 아니라 기능을 나눔)

Cooperating process는 데이터 교환(송수신)을 위해서 IPC(프로세스간 통신)가 필요함.

2가지 IPC 모델

- 1) **Shared-memory** 공유 메모리 : PA와 PB가 약속된 특정 메모리를 공유  
    kernel 영역을 거치지 않음. 속도가 빠름. but, 동기화 문제를 유저 프로세스가 직접 관리해야 함.
- 2) **Message-passing** 메시지 패싱 : PA가 PB에게 메시지를 직접 던지는 게 아니라 OS가 제공하는 **통로** 사용  
    kernel 영역을 사용. 속도가 다소 느릴 수 있음.

### Producer-Consumer Problem 생산자-소비자 문제

**생산자 Producer** 프로세스는 **소비자 Consumer** 프로세스에 의해 소비되는 정보를 생산함.

e.g. 컴파일러 : **컴파일러**가 어셈블리 코드를 생산하면, **어셈블러**가 그 코드를 가져와서 소비함.

e.g. 웹서버와 클라이언트 : **웹서버**는 HTML 파일이나 이미지같은 콘텐츠를 **생산**. **웹 브라우저**는 이를 읽어서 **소비**.

### IPC - Shared Memory 공유 메모리

: 통신을 원하는 프로세스 사이에 공유되는 메모리 영역

**유저 프로세스**가 통신을 제어함. NOT by OS

메모리를 공유할 때 어떻게 동기화 **Synchronize**하는지에 대한 메커니즘을 제공을 해야 함.

-> 여러 프로세스가 동시에 접근하면 데이터가 꼬일 수 있어, 순서를 맞추는 것이 중요함

### Bounded-Buffer

**in** : 앞으로 새로운 데이터를 넣을 자리를 가리키고 있음.

**out** : 데이터를 소비할 자리를 가리키고 있음.

in==out이면 비어있음. in보다 out 바로 뒤면?

// 공유될 데이터 구조 정의

#define BUFFER\_SIZE 10 // 버퍼의 크기를 10개로 고정함

typedef struct {

... // 실제 데이터 내용 (구조체)

} item;

item **buffer**[BUFFER\_SIZE]; // 데이터를 담을 10칸짜리 배열(공유 메모리 공간)

int **in** = 0; // 생산자가 다음에 넣을 위치 (초기값 0)

int **out** = 0; // 소비자가 다음에 꺼낼 위치 (초기값 0)

//Producer Process 생산자 프로세스

item next\_produced;

while (true) {

/\* produce an item in next\_produced \*/

while (((in + 1) % BUFFER\_SIZE) == out)

//버퍼가 꽉 찼으면 대기

; /\* do nothing \*/

//근데 in=9 out=0인 경우

buffer[in] = next\_produced;

in = (in + 1) % BUFFER\_SIZE;

//원형으로 구현

}

//Consumer Process 소비자 프로세스

item next\_consumed;

while (true) {

while (in == out)

// 소비할 게 없으면 대기

; /\* do nothing \*/

next\_consumed = buffer[out];

out = (out + 1) % BUFFER\_SIZE;

//원형으로 구현

/\* consume the item in next\_consumed \*/

}

## IPC - Message Passing 메시지 전달

: OS는 협력 프로세스를 위한 수단을 제공. -> Message-passing 시설을 통해서 서로 통신할 수 있도록 함.

같은 주소 공간을 공유하지 않고, 프로세스들이 통신하고 동작으로 동기화할 수 있게 함.

IPC 시설 facility는 두 가지 연산을 제공함.

**send(message)**                    생산자 프로세스에서 사용하는 함수

**recieve(message)**                소비자 프로세스에서 사용하는 함수

메시지 크기는 고정 또는 가변.

//Producer Process 생산자 프로세스

message next\_produced; // 생산된 메시지를 담을 변수

while (true) {

/\* next\_produced에 아이템을 생산함 \*/

**send**(next\_produced); // 생산된 메시지를 전송함

}

//Consumer Process 소비자 프로세스

```
message next_consumed;           // 소비될 메시지를 담을 변수
while (true) {
    receive(next_consumed);       // 메시지를 수신함
    /* next_consumed에 있는 아이템을 소비함 */
}
```

프로세스 P와 Q가 통신을 하려면 **통신 링크 Communication link**를 설정해야 함.  
**send()/receive()**를 통해서 메시지를 교환함

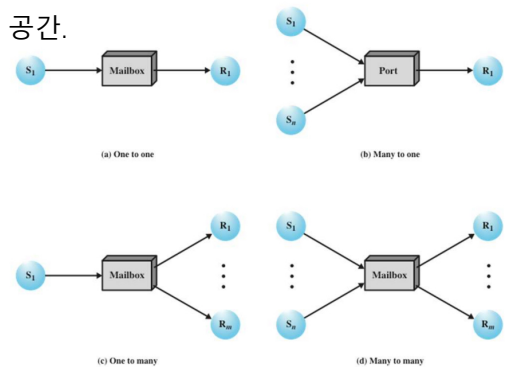
- 다양한 방식으로 구현될 수 있음. - 물리적 구현에 관해서는 다루지 않음. 논리적 구현에 대해서 다름.
- |                                                   |                   |               |
|---------------------------------------------------|-------------------|---------------|
| <b>Direct</b> 직접 - <b>Indirect</b> 간접 통신          | 수신자 명확히 지정(직접)    | 메일박스/포트(간접)   |
| <b>Synchronous</b> 동기 - <b>Asynchronous</b> 동기 통신 | 상대가 받을때까지 기다림(동기) | 던지기만 함(비동기)   |
| <b>Automatic</b> 자동 - <b>Explicit</b> 명시적 버퍼링     | 버퍼를 시스템이 자동관리(자동) | 프로그래머 지정(명시적) |

**Direct Communication - Naming 직접통신**  
: 통신의 수신자(Recipient), 발신자(Sender)를 명시적으로 지정  
**send(P, message)** - 프로세스 P에게 메시지를 보냄  
**recieve(Q, message)** - 프로세스 Q로부터 메시지를 받음

Directy Communication의 특징 :  
link가 **자동으로 설정됨**.(OS가 보통 자동으로 만듦)  
하나의 link는 정확히 2개의 프로세스와 연관됨. 1대1 연결을 함  
각각의 프로세스 쌍 사이에는 정확히 하나의 링크만 존재.

**Indirect Communication - Naming 간접통신**  
메시지가 **mailbox 메일박스/port** 포트로 전달됨.

**Mailboxes aka Ports** : 프로세스에 의해 메시지가 놓일 수 있는 곳.  
메시지가 제거(removed)될 수 있는 곳. - 메시지를 꺼내갈 수 있는 공간.  
각각의 mailbox는 **unique한 id**를 가지고 있음  
프로세스들은 메일박스를 **공유하는 경우에만 통신**할 수 있음.



통신 **Primitives**(기본 연산)  
**send(A, message)** - 메일박스 A로 메시지를 보낸다  
**recieve(A, message)** - 메일박스 A로부터 메시지를 받는다

Communication link의 특징 :  
두 프로세스 멤버가 **공유된 mailbox를 가질 때만** 둘 사이에 link가 설정됨.  
하나의 링크는 **두 개 이상**의 프로세스와 연관될 수 있다.  
통신하는 각 프로세스 쌍 사이에는 **여러 개의 서로 다른 링크가 존재**할 수 있음. 서로 다른 서비스마다 메일박스를 생성해서 통신을 할 수 있음.

- |             |                                     |
|-------------|-------------------------------------|
| 1대1         | 다대1(웹서버, 포트, 수많은 클라이언트가 하나의 서버로 접속) |
| 1대다(TV, 방송) | 다대다(단톡방)                            |

OS는 프로세스에게 아래와 같은 메커니즘을 제공해야 함  
새로운 mailbox(port) **생성**  
mailbox를 통해서 메시지 **송신/수신**  
mailbox **삭제**

mailbox는 프로세스에 의해서 소유될 수도 있고, OS에 의해서 소유될 수도 있음.

**프로세스 소유 時:** mailbox를 소유한 프로세스가 종료되면, 해당 mailbox 사라짐.

**OS 소유 時 :** OS에 소유된 mailbox는 자체적인 existence(존재권)를 가짐 - 독립적, 종속X e.g. 프린트 큐

## Synchronization 동기화

blocking 이거나 non-blocking일 수 있음

**Blocking => Synchronous 동기화 방식**

**Blocking send :** sender는 메시지가 receiver에서 수신될 때까지 block됨(기다림)

**Blocking receive :** receiver는 메시지가 도착할 때까지 block됨(기다림)

**Non-Blocking => Asynchronous 비동기화 방식**

**Non-Blocking send :** sender가 메시지를 계속 보냄. 수신 여부 상관 없음.

**Non-Blocking receive :** 메시지 있으면 유효한 메시지받고, 없으면 NULL 메시지를 받음. 기다리지 않음.

send와 receive를 다른 방식으로 해서 조합을 할 수도 있다.

**Rendezvous :** send와 receive 모두 blocking인 경우! 강한 결합, 강한 만남.

## Buffering 버퍼링

link에 붙어있는 메시지 큐

3가지 방법으로 구현 가능

1. Zero Capacity : 용량이 없음. 링크에 메시지가 대기할 수 없음. -> 무조건 받을 준비 될때까지 기다려야!  
-> Rendezvous 강제
2. Bounded Capacity : 유한한 용량. n개의 메시지를 담을 수 있는 한정된 길. 꽉 차면 대기해야 함.
3. Unbounded Capacity : 무한 용량 : 무한 대기 가능. sender는 절대 wait 하지 않음

---

## IPC 시스템 예제 - POSIX

POSIX에서는 메모리를 공유한다. memory-mapped files에 의해 구성됨.

공유 메모리 영역을 특정 파일과 연관시킴.

1. Shared memory 객체를 생성.

`shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);`

name : 공유메모리 이름

O\_CREAT는 없으면 만들라는 뜻.

O\_RDWR 읽기/쓰기 권한을 가지고 열여라

0666 권한 설정

2. 객체의 크기를 바이트 단위로 설정.

`ftruncate(shm_fd, 4096);` // 4096 바이트 4Kb 공간 확보

3. 확보한 공유메모리 파일을 실제 프로세스 메모리 주소와 연결

`mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);`

0 : 주소. 주소가 0이면 OS가 알아서 적당한 공간

SIZE : 크기 byte. 아까 할당한 만큼. 4KB 적당히 해야함.

PROT\_READ | PROT\_WRITE : 메모리 영역 보호 정도. 읽기+쓰기 모두 가능.

MAP\_SHARED : 이 메모리의 변경 내용을 다른 프로세스와 공유할까?

shm\_fd : 1단계에서 만든 공유메모리 식별자

0 : 매핑을 시작할 위치. 0은 처음부터 매핑하겠다!

## IPC Producer 생산자

```
int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);

    return 0;
}
```

<< OS 라는 문자열을 name이라는 포인터 변수에 저장

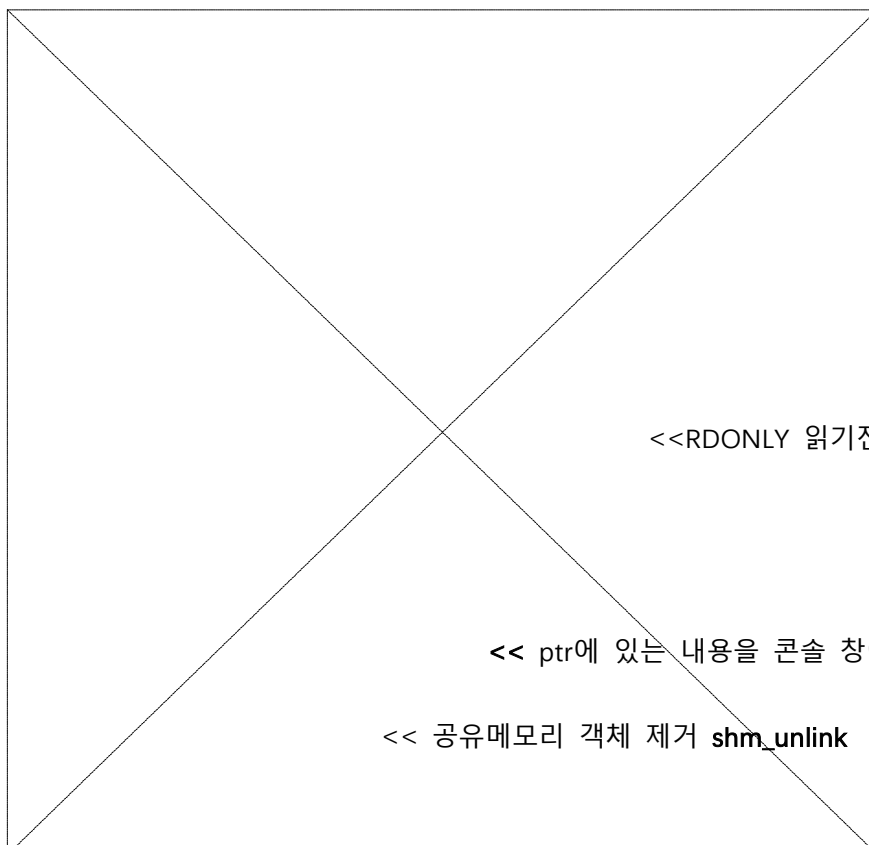
<< OS라는 이름의 공유메모리 객체 생성  
O\_CREAT 없으면 생성 | O\_RDWR 읽/쓰 권한

<< 생성된 공유메모리의 크기를 SIZE(4096byte) 만큼 설정.

<< 생성한 공유 메모리 객체를 실제  
프로세스의 가상 메모리에 매핑!

<< "Hello"를 메모리에 쓰기  
<< "Hello"의 길이만큼 포인터를 건너뛰기(점프!)  
<< 그 위치에서 다시 "World"를 써서 데이터 완성

## IPC Consumer 소비자



<< RDONLY 읽기전용으로 OS 라는 이름의 공유메모리 열기

<< 공유 메모리 객체를  
실제 프로세스의 가상 메모리에 매핑!

<< ptr에 있는 내용을 콘솔 창에 출력

<< 공유메모리 객체 제거 shm\_unlink

## Pipe 파이프

두 프로세스가 통신할 수 있는 통로(conduit) 역할을 함.  
초기 UNIX 시스템의 프로세스 통신 방법

### 문제 Issues

파이프가 **양방향(Bidirectional)**하게 통신을 허용하는가? 아니면 **단방향(Unidirectional)**하게 통신을 허용하는가?  
만약 양방향 통신을 허용한다면, **half-duplex(반이중)**인가? 아니면 **full duplex(전이중)**인가?  
프로세스 사이에 **부모-자식**같은 관계가 존재해야 하는가?  
파이프가 **네트워크를 통해** 통신할 수 있는가? 아니면 통신하는 프로세스가 **같은 머신 안에** 있는가?

## Ordinary Pipes vs Named pipes

### Ordinary Pipes 일반 파이프

생성한 프로세스 바깥에서는 접근할 수 없음. 그 프로세스를 만든 프로세스와 관련된 프로세스만 사용 가능  
부모 프로세스가 파이프를 만들고, 자식 프로세스는 통신할 때 부모가 만든 것 사용(자식은 사용 가능!)  
부모 - 자식 간은 같은 Pipe를 공유

### Named Pipes 이름 있는 파이프

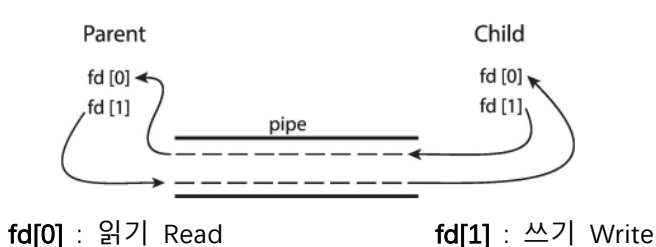
이름이 있기 때문에 부모-자식 관계가 아니어도 접근 가능. 외부에서도 접근 가능!  
서로 관련없는 프로세스끼리도 사용 가능.

## Ordinary Pipes

두 프로세스가 표준 생산자-소비자 스타일로 통신함.

생산자는 파이프의 한쪽 끝에 데이터를 씀. **write-end**  
소비자는 파이프의 다른쪽 끝에서 데이터를 읽음. **read-end**

**Unidirectional 단방향이다!** 데이터가 **한 방향으로만** 흐른다! 한 파이프로 양쪽 방향으로 통신하는 것이 불가  
**양방향(2-way)** 통신이 필요하다면, **2개의 파이프**가 필요함. 각각은 다른 방향으로 흐름



UNIX에서는 함수를 사용해서 ordinary pipe를 만든다.

```
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1
```

```
int main(void)
{
```

```
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];      << fd[0] read-end      fd[1] write-end
    pid_t pid;
```

write-end에 "Greetings" 쓰기>>

다 썼으면 닫기! >>

read-end 안 쓰니까 닫기 >>

메시지 읽고 출력하기 >>

```
/* create the pipe */
if (pipe(fd) == -1) {
    fprintf(stderr, "Pipe failed");
    return 1;
}

/* fork a child process */
pid = fork();

if (pid < 0) { /* error occurred */
    fprintf(stderr, "Fork Failed");
    return 1;
}

if (pid > 0) { /* parent process */
    /* close the unused end of the pipe */
    close(fd[READ_END]); << read-end는 안 쓰니까 닫기!

    /* write to the pipe */
    write(fd[WRITE_END], write_msg, strlen(write_msg)+1);

    /* close the write end of the pipe */
    close(fd[WRITE_END]);
}
else { /* child process */
    /* close the unused end of the pipe */
    close(fd[WRITE_END]);

    /* read from the pipe */
    read(fd[READ_END], read_msg, BUFFER_SIZE);
    printf("read %s", read_msg);

    /* close the read end of the pipe */
    close(fd[READ_END]); << 다 썼으면 닫기

    return 0;
}
```

## Named Pipe 이름 파이프

Ordinary pipe보다 더 파워풀함.

### Bidirectional 양방향 통신

프로세스 통신을 하기 위해서 부모-자식 관계가 필요하지 않음.

여러 프로세스가 named pipe를 통신에 사용할 수 있음.

## Client-Server 클라이언트-서버 시스템에서의 통신

클라이언트- 서버 통신에서는 2가지가 필요함

1. Sockets (소켓)
2. Remote Procedure Calls(RPC)

## Sockets 소켓

: 통신을 위한 endpoint. e.g. 내 컴퓨터의 어떤 프로그램, 상대 컴퓨터의 어떤 프로그램

소켓 : IP 주소 + 포트 번호 e.g. 161.25.19.8:1625 << 여기서 port는 1625 호스트 IPV4 주소는 161.25....

통신은 한 쌍의 소켓에서 이루어 짐

1024번 이하는 well-known이어서 일반 유저가 사용하지 않는 것이 좋음.

80 : HTTP 443 : HTTPS 21 : FTP 25 : SMTP 22 : SSH

127.0.0.1 루프백(Loopback) 프로세스가 실행 중인 바로 그 시스템을 가르키기 위해서 사용됨. 내 자신!  
테스트용으로 사용됨.

## Java에서 소켓

소켓의 3가지 타입

- |                                         |                               |
|-----------------------------------------|-------------------------------|
| 1. 소켓 클래스 Socket Class :                | TCP 연결지향(Connection-oriented) |
| 2. 데이터그램 소켓 클래스 DatagramSocket Class :  | UDP 연결 없음(Connectionless)     |
| 3. 멀티캐스트 소켓 클래스 MulticastSocket Class : | 데이터를 여러 수신자에게 보낼 수 있음         |

## 생산자

```
import java.net.*;
import java.io.*;

public class DateServer
{
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);           << 6013번으로 소켓을 만들었다!

            /* now listen for connections */
            while (true) {
                Socket client = sock.accept();                 << accept() 함수를 통해서 클라이언트가
                                                                접속할 때까지 무한대기.

                PrintWriter pout = new
                    PrintWriter(client.getOutputStream(), true);    << 클라이언트에게 문자열
                                                                보내는 출력기 만들기

                /* write the Date to the socket */
                pout.println(new java.util.Date().toString());      << 현재 날짜를 문자열로 보냄

                /* close the socket and resume */
                /* listening for connections */
                client.close();                                << 클라이언트와 연결 닫음!
            }
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```



소비자

```
import java.net.*;
import java.io.*;

public class DateClient
{
    public static void main(String[] args) {
        try {
            /* make connection to server socket */
            Socket sock = new Socket("127.0.0.1",6013);
            InputStream in = sock.getInputStream();
            BufferedReader bin = new
                BufferedReader(new InputStreamReader(in));

            /* read the date from the socket */
            String line;
            while ( (line = bin.readLine()) != null)
                System.out.println(line);

            /* close the socket connection*/
            sock.close();
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```

<< "내 컴퓨터 6013 포트에 접근  
루프백!  
<< 바이트 입력을 문자로 바꾸기  
<<null이 아닐 때까지 읽기  
<< 소켓 닫기(release 하기)

>> 서버 프로세스와 클라이언트 프로세스가 각각 실행되어야 통신 가능!

#### Remote Procedure Calls 원격 프로시저 호출

원격 서비스의 가장 흔한 형태 중 하나.

클라이언트가 "마치 로컬에서 하듯이" 원격 호스트의 프로시저를 호출함(involve)

#### RPS System (Remote Procdeure Call)

RPC 시스템은 클라이언트 쪽에 stub을 제공함. 통신이 일어나도록 하는 세부사항을 숨김

※ stub : 대리인, 중개인...

각각의 원격 프로시저마다 별도의 stub이 존재함.

클라이언트가 원격 프로시저를 호출하면, RPC 시스템은 그 원격 프로시저에 맞는 적절한 stub을 호출.

클라이언트 측 stub은 서버의 포트를 찾고, 매개변수를 마샬링(marshall)한다

※marshal : 매개변수를 네트워크에 보낼 수 있는 표준 형식으로 포장하는 것.

-> 그 애한테 맞게끔 바꿔서 던져줘야 함. 통일된 방식으로 바꿔주는 것..

클라이언트와 서버 기계 사이의 데이터 표현 문제 차이를 해결하기 위해서 marshallling을 해 주어야 함!

e.g. 엔디언 표기가 다를 수 있음! -> 따라서 marshallling을 해 주어야 한다.

어떤 컴퓨터는 빅 엔디언 12 34 56 78    어떤 컴퓨터는 리틀 엔디언 78 56 34 12 등등...

서버 측 stub는 이 메시지를 받아서, marshal된 매개변수를 unpack하고, 서버에서 프로시저를 수행한다.

## 5주차 Chapter 3: Threads & Concurrency

### 프로세스의 2가지 특징

#### 1. Resource Ownership 자원 소유

프로세스는 프로세스 이미지와 다른 자원들(CPU, I/O 장치, 파일들)을 담기 위한 가상 주소 공간을 포함함  
OS는 프로세스들 사이에 원하지 않는 간섭이 일어나지 않도록 보호를 해야 함.

#### 2. Scheduling/Execution 스케줄링/실행

한 프로세스의 실행은 번갈아 가면서(Interleaving) 실행될 수 있음.  
프로세스는 실행 상태(Running, Ready 등)와 디스패칭 우선순위를 가진다.  
프로세스가 CPU 스케줄링의 기본 대상이다

-> 이 두 특성은 서로 독립적이며, 운영체제에 의해 독립적으로 다루어질 수 있다.

디스패칭의 단위는 스레드(thread) 또는 경량 프로세스(lightweight process)라고 부름  
자원 소유의 단위는 프로세스(process) 또는 태스크(task)라고 부른다.

### Single Threaded Approach

프로세스마다 하나의 실행 스레드만 있는 것을 single-threaded approach라고 한다.  
예전에는 스레드라는 개념이 없었음. -> 프로세스 안에 여러 실행 흐름을 둔다는 것이 일반적이지 않았음  
e.g. MS-DOS는 오직 하나의 사용자 프로세스만 지원  
e.g. 초기 UNIX는 여러 사용자 프로세스를 지원 -> 하지만 실행 흐름은 여전히 하나

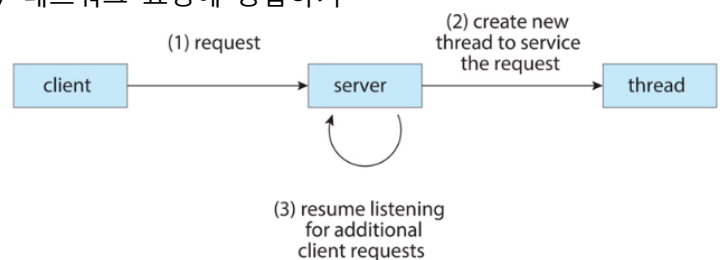
### Multithreading 멀티스레딩

하나의 프로세스 내부에서 여러 개의 동시적인 실행 경로를 지원하는 능력  
e.g. 자바 실행 환경(Java run-time environment) 하나의 프로세스 안에 여러 스레드가 있는 시스템.  
e.g. 윈도우, 솔라리스, 리눅스는 multiple process를 허용, 각각의 프로세스는 여러 스레드를 지원.  
-> 대부분의 현대 응용프로그램은 멀티스레딩 방식이다. 스레드는 앱 내부에 속해 있음.

응용 프로그램 내부의 여러 작업은 각각 별도의 스레드로 구현될 수 있다.

e.g. 화면 갱신 / 데이터 가져오기 / 맞춤법 검사 / 네트워크 요청에 응답하기

#### 커널에서도 멀티스레드를 지원함!



서버가 요청을 받으면 새로운 스레드를 생성해서 응대,  
그리고 난 후에 listening을 계속 함. 멀티스레딩!

### Process 프로세스

멀티스레드 환경에서 프로세스는 '자원 할당의 단위이자 보호의 단위'로 정의된다

-> 즉, 프로세스는 CPU를 직접 독점하는 게 아니라, 보호되고 통제된 방식으로 CPU를 할당받는다.

1. 프로세스 이미지를 담고 있는 가상 주소 공간

2. 다음 자원들에 대한 보호된 접근. 프로세스

프로세스, 다른 프로세스(interprocess 통신 시), 파일, I/O 요청(장치와 채널)

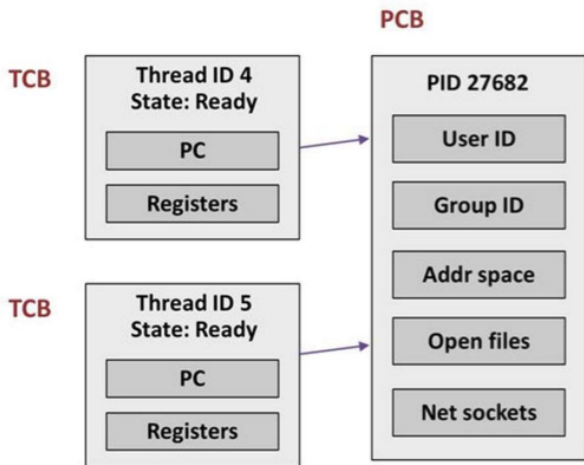
thread = CPU에서 실제로 실행되는 단위

process = 메모리와 자원을 보유하고 보호받는 단위

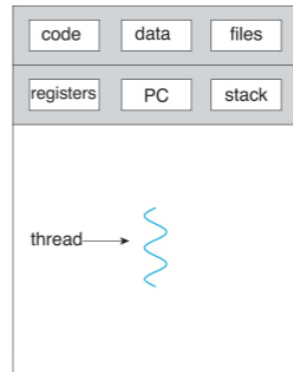
## 프로세스 안에서 스레드의 특성

1. **실행 상태**(Running, Ready 등등...) - 각각의 스레드는 각자의 상태가 있음.
2. **context 정보**를 저장 : 작동하지 않을 때는 스레드의 context 정보 저장해놔야 함. 깨어나고 멈추기 위해서.
3. **실행 스택**을 가짐. - 각 스레드는 자기만의 스택을 가짐. 함수 호출순서, 지역, 매개, 리턴가 각 스레드별로 관리되어야 하기 때문

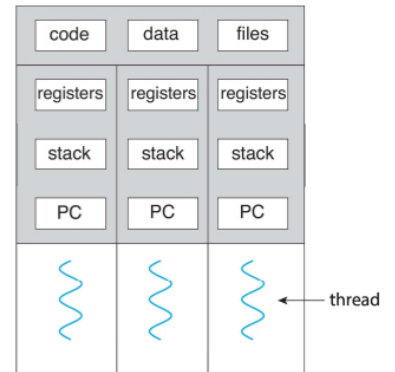
## Process creation is heavy-weight while thread creation is light-weight



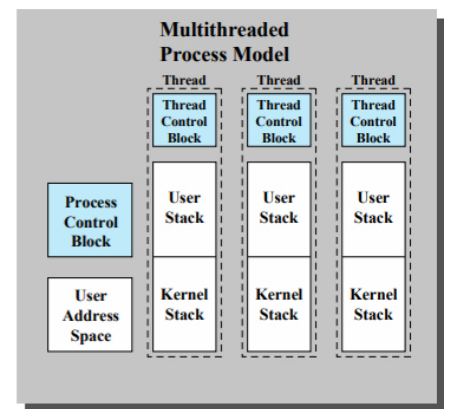
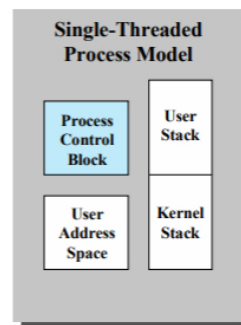
PCB에는 PID가 있고,  
각 스레드도 TCB에 Thread ID가 있음.



single-threaded process



multithreaded process



프로세스 생성은 무겁고(thread보다 비용 큼)  
스레드는 프로세스 자원을 공유하므로 더 가볍다

## Multithreaded programming의 장점

### Responsiveness 응답성

프로세스의 한 부분이 block이 되어도 다른 부분의 계속 실행할 수 있음. 별개로 움직일 수 있다.  
특히, 사용자 인터페이스(UI)에서 중요함. 한 스레드는 데이터 받아오기, 다른 스레드는 화면 표시 등등..  
-> 멈췄다는 느낌이 안 남!

### Resource Sharing 자원 공유

스레드들은 프로세스의 자원을 공유한다.  
코드, 데이터, 주소 공간(address space), 파일 등 공유  
shared memory나 message passing에 비해서 쉽다! 자원을 공유하기 때문.

### Economy 경제적인

process를 생성하는 것보다 싸다!  
스레드 전환은 프로세스 Context Switch보다 오버헤드가 더 낮다.  
-> 자원을 바꿀 필요가 없기 때문. 레지스터, 스택 PC 등만 바꿔주면 됨!

### Scalability 확장성

프로세스는 멀티코어 구조를 활용할 수 있다.  
멀티스레드면 각각의 작업을 서로 다른 코어에서 동시에 실행할 수 있어서 훨씬 효율적이다!

## Thread 스레드

스레드를 지원하는 운영체제에서는 스케줄링과 디스패칭이 **스레드 단위**로 이루어진다.

실행과 관련된 대부분의 상태 정보는 **스레드 수준의 자료구조**에 유지됨.

-> TCB에 들어 있음. 상태정보(Running...), PC, 레지스터, stack 관련 정보

프로세스 안의 모든 스레드에 영향을 미치는 동작들 (예외)

**프로세스 중단(suspend)** : 프로세스가 중단되면 그 프로세스의 모든 스레드가 중단됨.

같은 주소 공간을 공유하기 때문

**프로세스 종료(terminate)** : 프로세스가 종료되면 그 프로세스의 모든 스레드가 종료됨.

## Thread Execution States 스레드 실행 상태

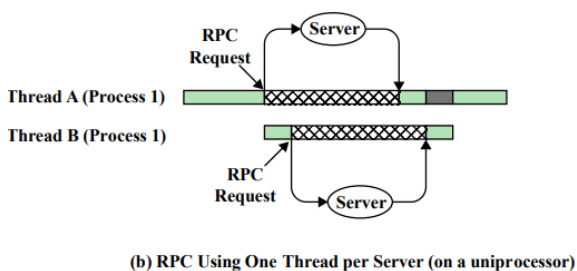
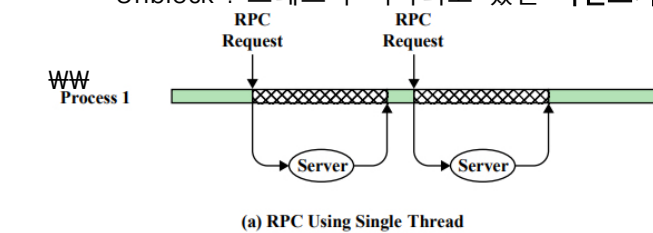
주요 상태 : Running, Ready, Blocked

주요 작업 :

Spawn : 프로세스 안의 한 스레드는 같은 프로세스 안에 또 다른 스레드를 생성할 수 있음.

Block : 스레드가 어떤 이벤트를 기다려야 할 때, 그 스레드는 blocked

Unblock : 스레드가 기다리고 있던 이벤트가 발생하면, 그 스레드는 **Ready** 큐로 이동한다.



Blocked, waiting for response to RPC  
 Blocked, waiting for processor, which is in use by Thread B  
 Running

Figure 4.3 Remote Procedure Call (RPC) Using Threads

## Thread Synchronization 스레드 동기화

여러 스레드의 활동을 동기화하는 것이 중요하다!

하나의 프로세스에 속한 모든 스레드는 같은 주소 공간과 다른 자원들을 공유한다.

→ 한 스레드가 자원을 변경하면, 그 변화는 같은 프로세스 안의 다른 스레드들에게 영향을 준다.

**Thread의 종류** 누가 스레드를 관리하는냐에 따라서 달라짐.

User Level Thread(ULT; 관리 by ULT 라이브러리)와 Kernel Level Thread(KLT; 관리 by 커널)로 나뉜다.

## User-Level Thread(ULT) 유저 레벨 스레드

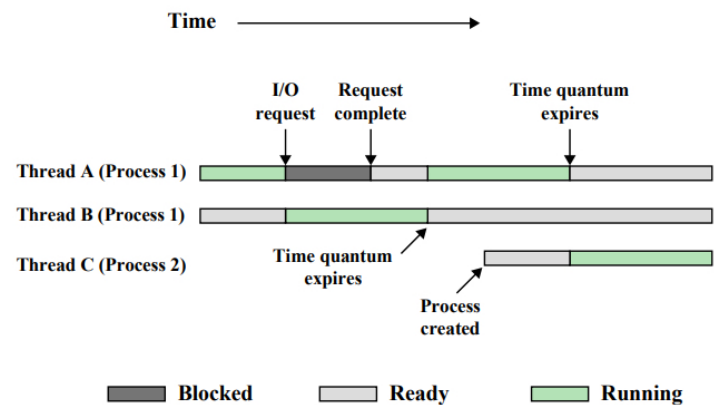
모든 스레드 관리는 응용프로그램(application)에 의해 이루어짐.

커널은 스레드의 존재를 알지 못한다! 커널한테는 내부 사정...

많은 ULT가 하나의 커널 스레드에 매핑된다.

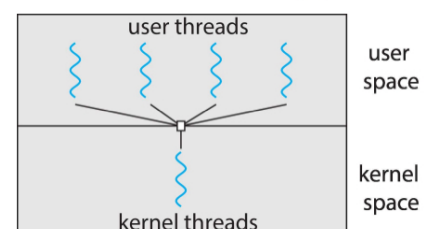
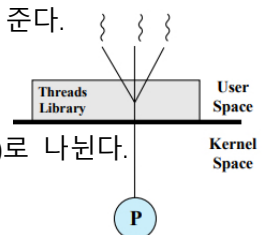
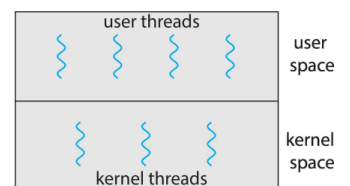
e.g. 예전에 Solaris Green Threads에서 사용함

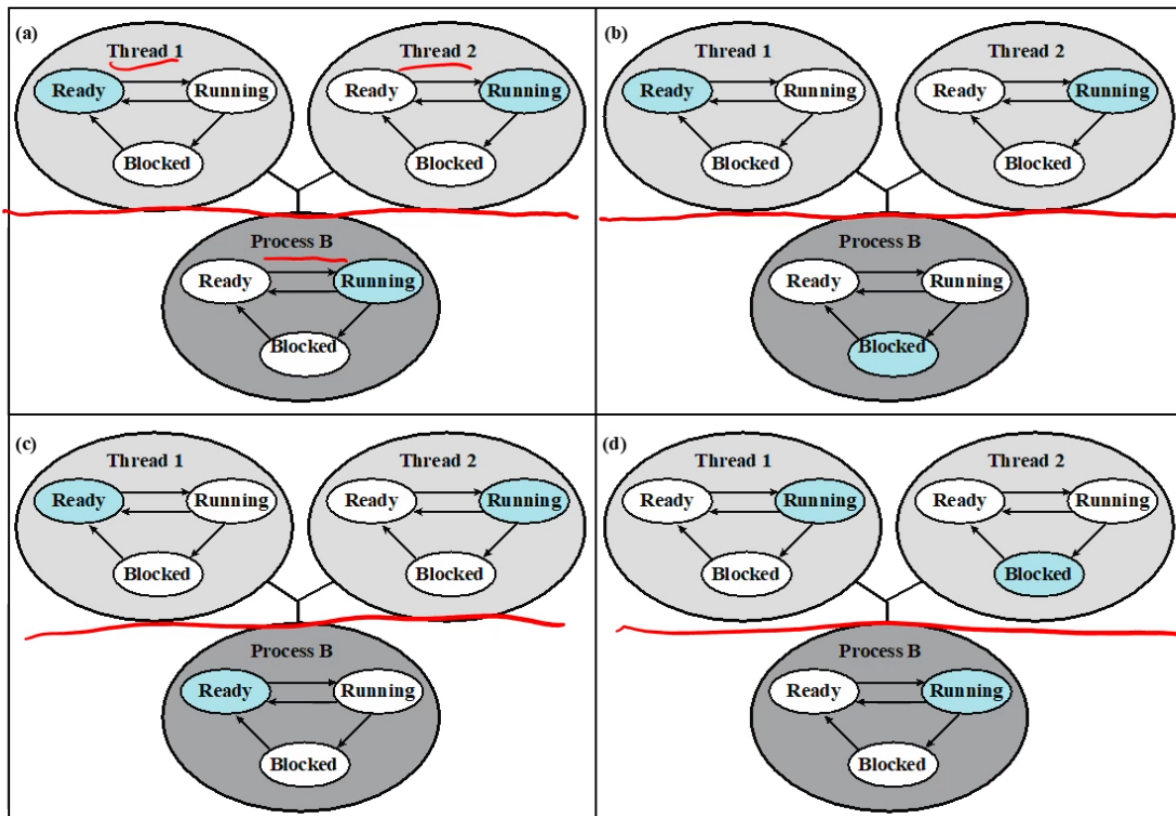
**MANY to ONE**



▲ 스레드 간 context switch도 일어나고 프로세서 간도 일어남

Figure 4.4 Multithreading Example on a Uniprocessor





(a)  
Thread 2가 ULT 차원에서 Running  
Process B도 커널 차원에서 Running  
→ 둘이 잘 맞아떨어지는 경우

(b)  
Thread 2는 ULT 차원에서 Running  
Process B는 커널 차원에서 Blocked  
왜냐면 ULT 중 하나가 I/O 요청이나 system call을 했기 때문  
→ 유저 쪽에서는 “돌아야 함”이라고 생각하지만  
커널이 프로세스 전체를 막아버려서 실제로는 못 돌

(c)  
Thread 2는 ULT 차원에서 Running  
Process B는 커널 차원에서 Ready  
→ 유저 쪽에서는 실행 차례를 정했지만  
커널이 아직 CPU를 안 줌

(d)  
Thread 1은 Running Thread 2는 Blocked  
Process B는 Running  
→ 프로세스 전체는 CPU를 받고 있고,  
그 안에서 유저 라이브러리가 Thread 1을 돌리는 중

(a) -> (b) -> (c) -> (a)

### ULT의 장점

Thread switching(스레드끼리 switch)에는 커널 모드 권한이 필요하지 않다!  
스케줄링을 앱에 맞게 특화할 수 있다. <- 앱이 알아서 라이브러리 스케줄링 알 수 있다.  
ULT는 어떤 OS에서도 실행될 수 있다. 라이브러리만 있으면 쓸 수 있다!

### ULT의 단점

일반적인 OS에서는 많은 시스템 콜이 blocking 방식이다.  
하나의 ULT가 시스템 콜을 수행하면 그 스레드만 block되는 것이 아니라, 그 프로세스 안의 모든 스레드가 함께 block된다.

순수 ULT 방식에서는 멀티스레드 응용프로그램이 multiprocessing의 이점을 활용할 수 없다.

-> 진정한 병렬 처리가 아님

커널은 한 번에 하나의 프로세스를 하나의 프로세서에만 할당하므로, 결과적으로 한 프로세스 안에서는 한 시점에 하나의 스레드만 실행될 수 있다.

## ULT의 단점을 극복하기 위한 방법

응용프로그램을 multiple threads가 아니라 **multiple processes**로 작성하는 것  
하나가 block 되어도 다른 프로세스에서 돌아갈 수 있음.

-> but, 그렇게 하면 multi thread의 주된 장점을 없애버린다.

-> process switch를 해야 하므로 오버헤드가 커지게 된다.

### Jacketing

: blocking system call을 **non-blocking system call**로 바꾼다.

스레드는 시스템 I/O 루틴을 직접 호출하는 대신, 앱 수준의 I/O **jacket routine**을 호출한다.

직접 blocking I/O를 하지 않고

중간의 jacket routine이 대신 처리하게 만드는 것

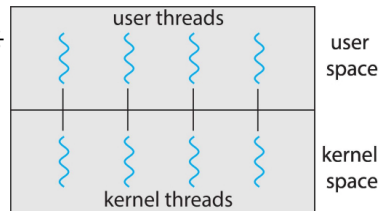
## Kernel-Level Threads(KLTs)

커널이 스레드를 관리함. OS가 직접 관리!

앱 레벨에서는 스레드를 관리하는 코드가 없음. 단지 커널의 스레드 기능을 쓰기 위한 API만 있다.

각각의 user-level thread는 각각의 kernel thread와 연결됨

e.g. 윈도우/리눅스/솔라리스 9 포함 이후버전은 이 방식 사용



(b) Pure kernel-level

### One-to-One 일대일

#### KLTs 장점

many-to-one보다 더 **동시성**이 높다(**concurrency** 높다)

커널은 같은 프로세스에 속한 여러 스레드를 여러 프로세서(CPU)에서 동시에 스케줄할 수 있다

프로세스 안의 한 스레드가 **block**되면, 커널은 같은 프로세스의 다른 스레드를 스케줄할 수 있다

#### KLTs 단점

같은 프로세스 내부에서 한 스레드에서 다른 스레드로 제어가 넘어갈 때도, 커널로의 모드 전환(mode switch)이 필요하다 -> **오버헤드가 든다.**

오버헤드 때문에 프로세스 당 스레드 수가 때때로 제한될 수 있다

| Operation(벤치마킹 종류)     | User-Level Threads | Kernel-Level Threads | Processes |
|------------------------|--------------------|----------------------|-----------|
| Null Fork(새로운 프로세스 생성) | 34                 | 948                  | 11,300    |
| Signal Wait(통신에서 기다림)  | 37                 | 441                  | 1,840     |

>> 커널이 개입하는 순간 오버헤드가 엄청 커짐. 시간이 오래 걸림

## Combined Approach

ULT커널 수준과 스레드를 적절히 섞는 방식

스레드 생성은 완전히 **사용자 공간(user space)**에서 이루어지며, 애플리케이션 내부 스레드들의 대부분의 스케줄링과 동기화도 사용자 공간에서 이루어진다. - ULT의 장점을 가져옴

여러 개의 ULT들이 몇 개의 KLT들에 매핑된다

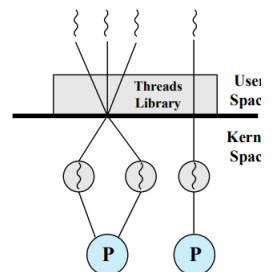
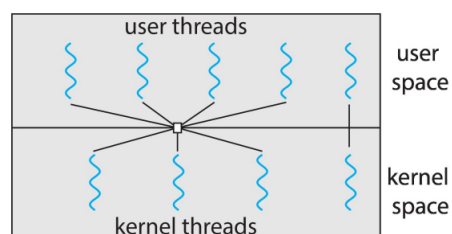
### Many-to-Many

ULT의 단점을 해결할 수 있다.

-> But, 실제로는 관리하기가 번거롭다.

Solaris 8 또는 그 이전 버전에서 사용하였음

Many-to-Many





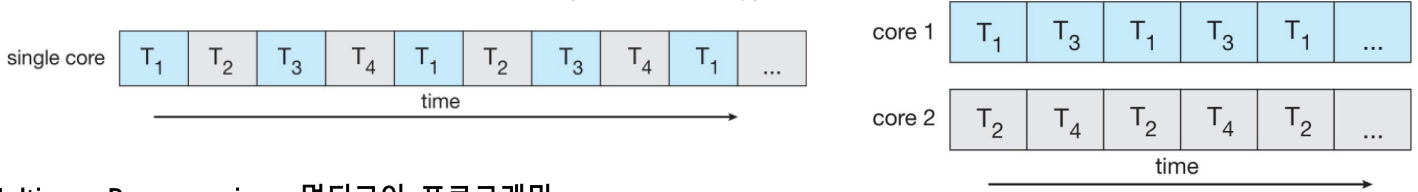
## Concurrency vs Parallelism

### 싱글코어 시스템에서 Concurrency

동시성은 둘 이상의 작업이 진행되도록 해 준다! **Interleaving**을 해서 조금씩 번갈아가면서 실행함.

### 멀티코어 시스템에서 Parallelism

병렬성은 시스템이 둘 이상의 작업을 동시에(simultaneously) 수행할 수 있음을 의미한다.



## Multicore Programming 멀티코어 프로그래밍

멀티코어 또는 다중 프로세서 시스템은 프로그래머에게 부담을 주며, 그 과제(challenge)는 다음과 같다.

### 1. Identifying tasks 작업 식별하기

별도의 작업으로 나눌 수 있는 부분을 찾아야 함.

작업들은 서로 독립적이어야 하며, 그래야 각각의 코어에서 병렬로 실행될 수 있다

### 2. Balance 균형

프로그래머는 각 작업이 비슷한 양과 비슷한 가치의 일을 하도록 해야 한다

코어별로 일을 고르게 나누어 줘야 한다.

### 3. Data Splitting 데이터 분할

데이터는 각 코어에서 실행될 수 있도록 나누어져야 한다

### 4. Data dependency 데이터 의존성

프로그래머는 데이터 의존성을 처리할 수 있도록 작업들의 실행이 동기화되게 해야 한다

어떤 작업이 다른 작업이 만든 데이터를 써야 할 수 있음

순서를 맞추기 위해 **synchronization(동기화)** 필요함

### 5. Testing and debugging 테스트, 디버깅

단일 스레드 애플리케이션보다 테스트와 디버깅이 더 어렵다

## Types of Parallelism 병렬성의 종류

### Data Parallelism 데이터 병렬성 :

같은 데이터의 부분집합들을 여러 코어에 나누어 배치하고, 각 코어에서 같은 연산을 수행하는 방식  
데이터를 쪼개서 다른 코어에 똑같은 일을 시키기

### Task Parallelism 작업 병렬성 :

스레드들을 여러 코어에 분배하고, 각 스레드가 서로 다른 연산을 수행하는 방식

같은 데이터를 주고 각 코어에서 다른 일을 시킴. -> e.g. 한 자료로 다른 통계 계산

## Amdahl's Law 암달 법칙

코어를 늘린다고 무한히 성능이 좋아질까? -> 아님.

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

S : 순차적인 부분의 비율(병렬화 할 수 없는 부분. 꼭 순차적으로 해야하는 부분의 비율)

N : 처리 코어의 개수

$$\begin{aligned} speedup &= \frac{1}{0.05 + \frac{0.95}{2}} & speedup &= \frac{1}{0.25 + \frac{0.75}{2}} & speedup &= \frac{1}{0.5 + \frac{0.5}{2}} \\ &= \frac{1}{0.05 + 0.475} = \frac{1}{0.525} \approx 1.9 & &= \frac{1}{0.25 + 0.375} = \frac{1}{0.625} = 1.6 & &= \frac{1}{0.5 + 0.25} = \frac{1}{0.75} \approx 1.33 \end{aligned}$$

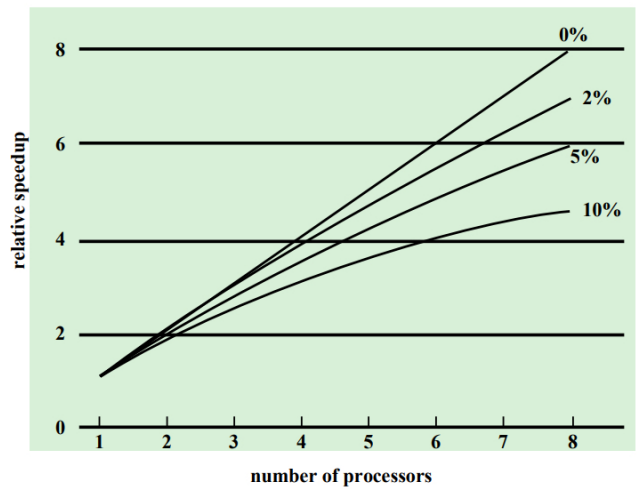
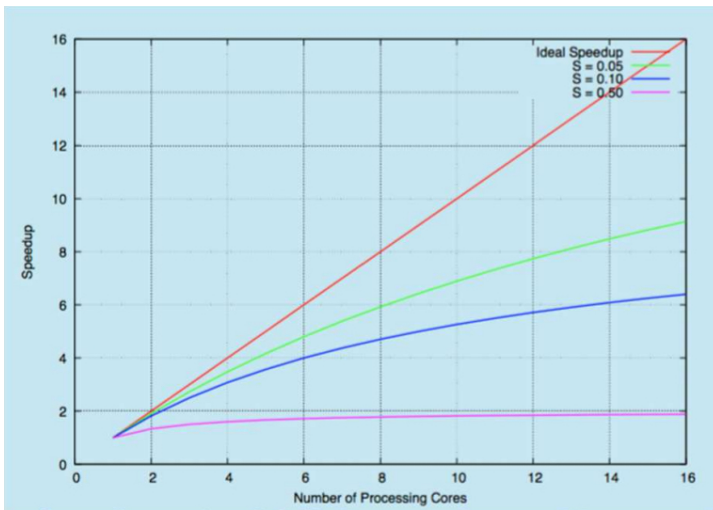
e.g. S = 5%, N = 2, Speedup = 1.9

e.g. S = 25%, N = 2, Speedup = 1.6

e.g. S = 50%, N = 2, Speedup = 1.3

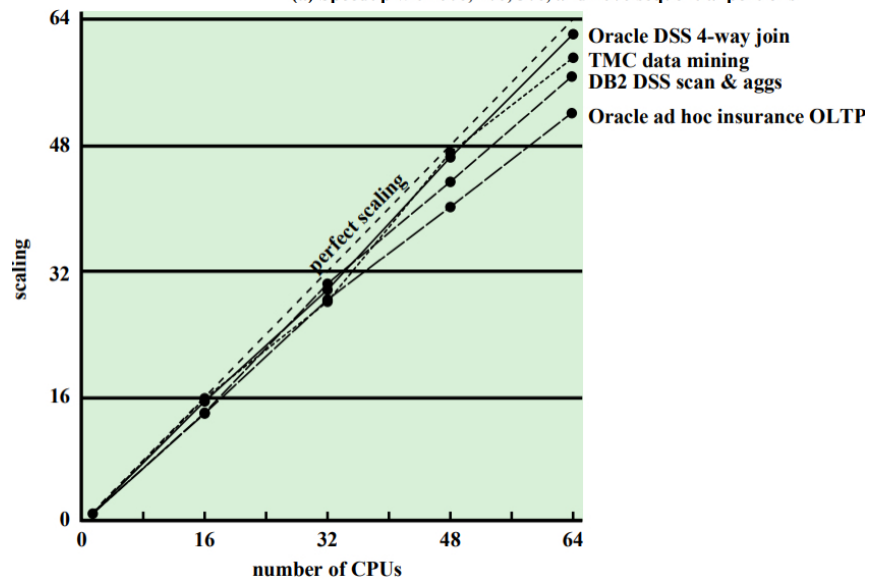
N이 무한대로 늘어나면 -> 1/S가 됨. 병렬 부분은 빨라지지만, 순차 부분(S)는 줄어들지 않음. 전체에는 한계 有





(a) Speedup with 0%, 2%, 5%, and 10% sequential portions

직선에 가깝도록 프로그래밍을 해야함  
DB시스템은 이에 근접함.



## Thread Libraries 스레드 라이브러리

스레드 라이브러리는 프로그래머에게 스레드를 생성하고 관리하기 위한 API를 제공함.

프로그래머가 스레드를 만들고, 실행시키고, 종료시키고, 동기화하고, 기다리는 작업을 하는 함수를 미리 만들어놓.

POSIX Pthread / Windows threads / Java threads가 유명함.

## POSIX 예제 1

```
#include <pthread.h>
#include <stdio.h>
```

<< pthread.h Pthread를 사용

```
#include <stdlib.h>
```

```
int sum; /* this data is shared by the thread(s) */ //전역변수 sum 생성
void *runner(void *param); /* threads call this function */ // 함수 원형.
```

```
int main(int argc, char *argv[])
{
```

```
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */
```

// tid 변수 선언

// 스레드 속성값 저장할 변수 선언

```
    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
```

// attr을 사용해서 속성값 설정. 안해도 됨

```
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
```

// 새 스레드를 만든다. (tid, 속성값, 실행시킬 함수, 전달할 매개변수)

```
    /* wait for the thread to exit */
    pthread_join(tid, NULL);
```

// tid에 해당하는 스레드가 끝날때까지 기다림!

```
    printf("sum = %d\n", sum);
```

//55가 출력됨

```
}
```

```

/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);    //argv[1]로 10이 들어왔다고 하면
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;                //1부터 10까지 sum에 더함. sum이 55가 됨

    pthread_exit(0);              //스레드 종료
}

```

## POSIX 예제 2

```

#include <pthread.h>
#include <stdio.h>

int value = 0;
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
    pid_t pid;
    pthread_t tid;
    pthread_attr_t attr;

    pid = fork();    //fork()를 해라!

    if (pid == 0) { /* child process */
        pthread_attr_t attr;
        pthread_create(&tid, &attr, runner, NULL);    //runner 함수를 실행하는 tid 스레드를 생성해라
        pthread_join(tid, NULL);                    //tid 스레드 끝날 때까지 기다리기
        printf("CHILD: value = %d", value); /* LINE C */
    }
    else if (pid > 0) { /* parent process */
        wait(NULL);    //자식 프로세스 끝날 때까지 기다리기
        printf("PARENT: value = %d", value); /* LINE P */
    }
}

void *runner(void *param) {
    value = 5;
    pthread_exit(0);
}

```



자식 프로세스(P1)에서는 value 5 출력, 부모 프로세스(P0)에서는 value 0 출력

## 7주차 Chapter 5: CPU Scheduling #1

### 프로세스 스케줄링의 종류

#### Multiprogramming 멀티프로그래밍

단일 프로세서(Uniprocessor) 시스템 안에서 여러 프로세스를 관리하는 것

#### Multiprocessing 멀티프로세싱

다중 프로세서(multiprocessor) 시스템 안에서 여러 프로세스를 관리하는 것

#### Distributed processing 분산 처리

여러 개의 분산된 컴퓨터 시스템에서 실행되는 여러 프로세스를 관리하는 것

-> Multiprogramming의 핵심은 스케줄링이다!

CPU는 하나인데 프로세스는 여러 개니까 OS가 이를 결정해야 함.

목표 : 항상 어떤 프로세스든 실행되고 있도록 하는 것, CPU 이용률을 최대화하는 것

### CPU I/P Burst Cycle

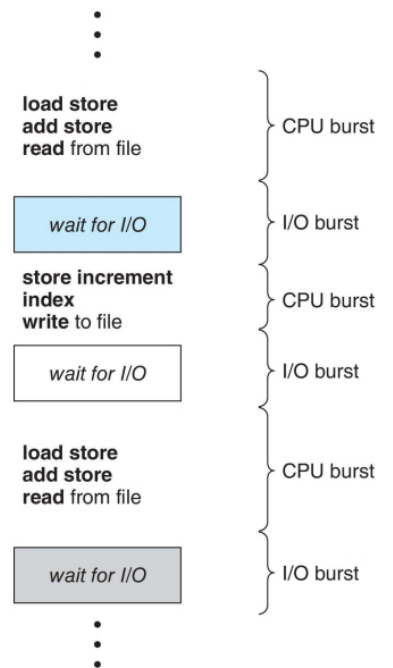
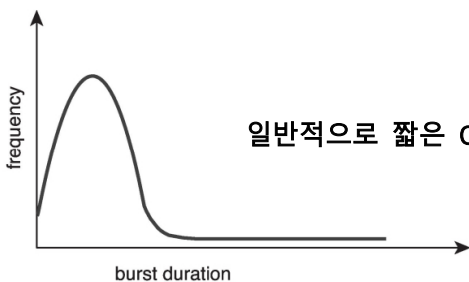
: 프로세스의 실행은 CPU 실행과 I/O wait의 반복 사이클로 이루어짐.

CPU burst뒤에는 I/O burst가 따라옴.

CPU burst : 프로세스가 실제로 CPU를 붙잡고 계산하는 시간 구간

I/O burst : 프로세스가 입출력(I/O)을 요청하고 기다리는 시간 구간

우리는 어떤 프로세스가 CPU burst가 많은지 I/O burst가 많은지 잘 알아야 함!



### Scheduling 스케줄링

: 대기열을 관리해서 대기 지연을 최소화하는 문제.

응답 시간, 처리량, 프로세서 효율 같은 시스템 목표를 만족하도록 프로세서가 실행할 프로세스를 할당하는 것

#### Long-term Scheduling

#### Medium-term Scheduling

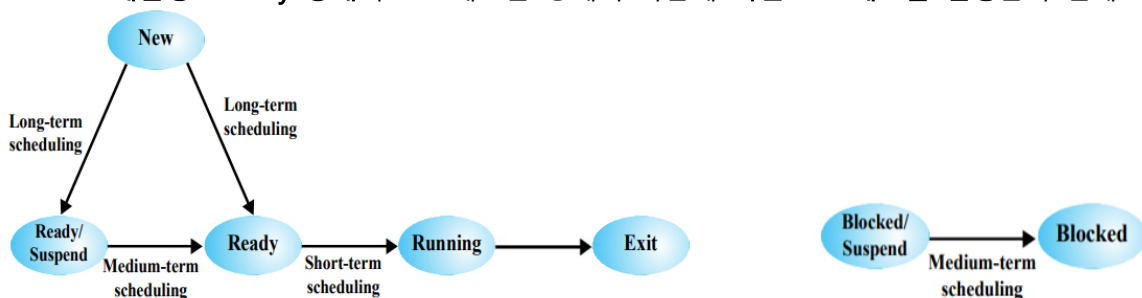
#### Short-term Scheduling으로 나누어짐.

Long-term 스케줄링과 Medium-term 스케줄링은 degree of multiprogramming와 관련된 성능 문제에 의해 결정됨.

Long-term 스케줄링 : 새 프로세스가 생성될 때 수행됨. 새 프로세스를 Ready Queue에 넣을까 말까?

Medium-term 스케줄링 : swapping 기능의 일부로 수행됨. 메모리에 계속 둘까, 잠깐 밖으로 뺄까?

Short term 스케줄링 : ready 상태의 프로세스들 중에서 다음에 어떤 프로세스를 실행할지 실제로 결정하는 것



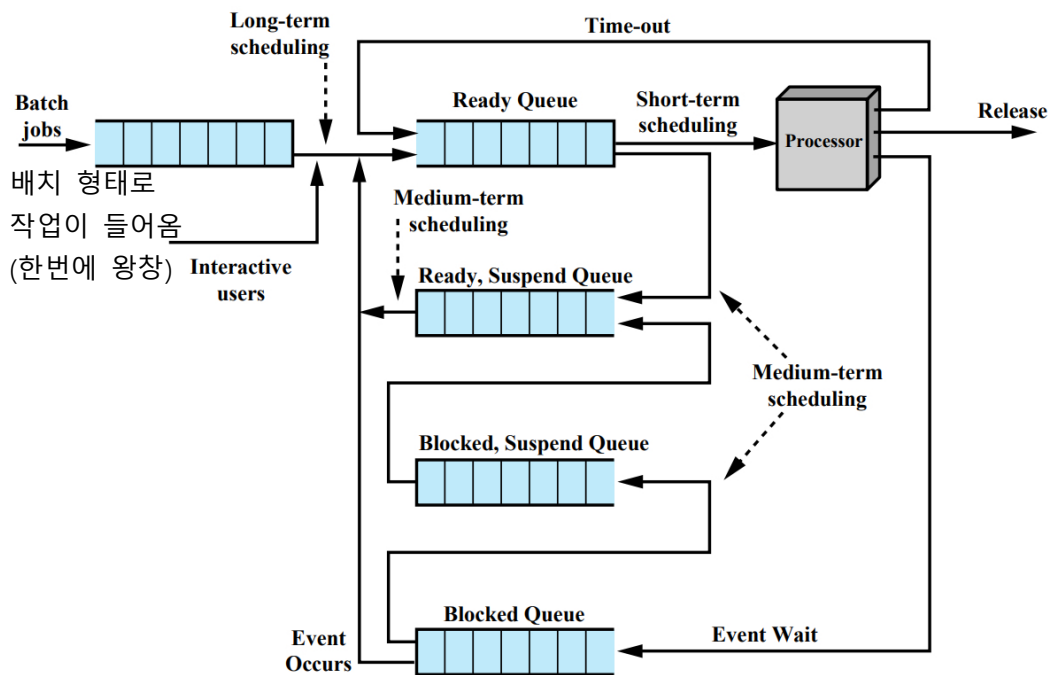


Figure 9.3 Queuing Diagram for Scheduling

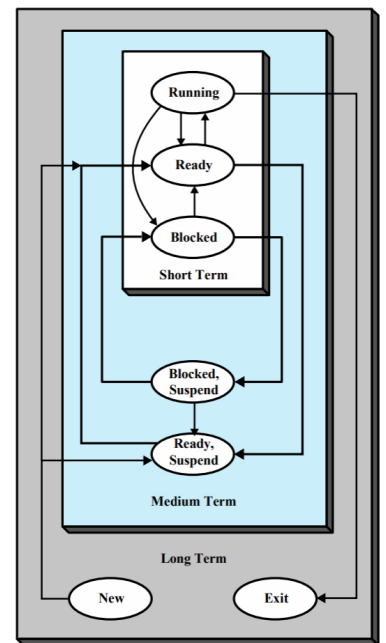


Figure 9.2 Levels of Scheduling

### Long-term Scheduler

어떤 프로그램(작업)들을 처리하기 위해 시스템에 받아들일지 결정

Controls the **degree of multiprogramming** 멀티프로그래밍의 정도를 조절한다 (얼마나 몇개나 받아들일까?)

프로세서의 **idle time**이 일정 기준(threshold)을 넘으면, long-term 스케줄러가 호출되어 **job**을 추가할 수 있다  
현재 시스템 안에 있는 프로세스들에게 만족스러운 서비스를 제공하기 위해, 그 **정도(degree)**는 제한될 수 있다

Long-term scheduler는 **job**을 **process**로 바꿔 시스템에 투입하는 역할

OS가 하나 이상의 추가 프로세스를 더 받아들일 수 있는 시점을 결정

언제?

어떤 **job**들을 받아들여서 프로세스로 바꿀지 결정

어떤걸?

-> **FSFC**(First Serve First Come, 먼저 온 순서대로), **Priority**(우선순위), 예상 실행 시간, I/O 요구량

### Medium-term Scheduler

swapping 기능의 일부

**Swap Out** : 멀티프로그래밍의 정도를 줄이기 위해, 프로세스를 **메인 메모리**에서 **밖으로** 내보낸다

한동안 활동하지 않은 프로세스

priority가 낮은 프로세스

page fault가 자주 일어날 때, hit을 잘 못할 때

메모리를 많이 차지하고 있는 프로세스 등등....

**Swap in** : 프로세스를 다시 **메모리 안으로** 들인다

더 많은 메모리가 사용 가능할 때

그 프로세스가 unblock되어 더 이상 자원을 기다리지 않게 되었을 때

### Short-term Scheduling (CPU Scheduling)

제일 빈번히 실행됨.

다음에 실행하게 될 프로세스를 결정하게 됨. 세밀한(fine-grained) 결정을 내림.

**ready queue**에 있는 프로세스들 중 하나를 선택해서, 그 프로세스에 **CPU 코어**를 할당한다

Queue는 **FIFO Queue**, **Priority Queue**, **Tree**, **Linked List** 등등 다양한 방법으로 구성될 수 있다.

## NonPreemptive vs Preemptive

selection function(scheduling algorithm)가 언제 실행될지를 정하는 것이다  
스케줄링 알고리즘이 언제 개입?

### Nonpreemptive 비선점형 양보 안함

일단 어떤 프로세스가 running 상태에 들어가면, 다음 경우까지 계속 실행된다.  
e.g. 그 프로그램이 종료되어 CPU를 내놓을 때까지  
e.g. I/O를 기다리거나 어떤 OS 서비스를 요청하기 위해 스스로 block될 때까지

### Preemptive 선점형 양보함

현재 실행 중인 프로세스는 중간에 인터럽트되어 Ready 상태로 옮겨질 수 있다  
e.g. 새로운 프로세스가 도착했을 때  
e.g. 인터럽트가 발생해서 block된 프로세스가 Ready 상태가 될 때  
e.g. 주기적으로 clock interrupt가 일어날 때  
-> Nonpreemptive보다 더 큰 오버헤드가 들 수 있지만,  
어떤 하나의 프로세스가 프로세서를 독점하는 것을 막아준다

## Dispatcher 디스패처

: short-term scheduler가 선택한 프로세스에게 CPU의 제어권을 넘겨준다.  
실제로 CPU를 그 프로세스에게 넘기는 역할!

현재 프로세스가 block될 수 있는 사건이 발생하거나, 현재 실행 중인 프로세스를 선점하고 다른 프로세스를 실행할 기회가 생길 때 dispatcher가 호출됨  
e.g. clock interrupt, I/O interrupt, 시스템콜, 시그널(동기화 이벤트)

Dispatch latency : : 한 프로세스를 멈추고 다른 프로세스를 실행시키기까지 dispatcher가 걸리는 시간

```
[root@ ~]# vmstat 1 3
```

| procs |   | memory |        |       |       | swap |    | io |    | system |     | cpu |    |     |    |    |
|-------|---|--------|--------|-------|-------|------|----|----|----|--------|-----|-----|----|-----|----|----|
| r     | b | swpd   | free   | buff  | cache | si   | so | bi | bo | in     | cs  | us  | sy | id  | wa | st |
| 0     | 0 | 48044  | 422212 | 28744 | 77560 | 0    | 0  | 0  | 3  | 0      | 1   | 0   | 0  | 100 | 0  | 0  |
| 0     | 0 | 48044  | 422220 | 28744 | 77568 | 0    | 0  | 0  | 0  | 66     | 60  | 0   | 0  | 100 | 0  | 0  |
| 0     | 0 | 48044  | 422204 | 28744 | 77568 | 0    | 0  | 0  | 0  | 159    | 162 | 2   | 0  | 98  | 0  | 0  |

vmstat 1 3 #시스템 상태를 1초마다 3번 보여줘!

>> 출력 결과에서 cs가 context switch의 횟수.

|                            |     |                       |
|----------------------------|-----|-----------------------|
| cat /proc/2166/status      |     |                       |
| voluntary_ctxt_switches    | 150 | //자발적 context switch  |
| nonvoluntary_ctxt_switches | 8   | //비자발적 context switch |

## Short term 스케줄링 기준(Criteria)

↑ CPU utilization : CPU가 가능한 한 계속 바쁘게 일하고 있나?

↑ Throughput : 처리량. 단위 시간당 완료되는 프로세스의 수. 얼마나 많은 일을 일정 시간 안에 끝냈는가?

↓ Turnaround time : 하나의 프로세스를 실행 완료하는 데 걸리는 전체 시간. 제출시간 -> 완료시간

↓ Waiting time : 프로세스가 ready queue에서 기다린 시간의 총합 ready큐에서 대기한 시간의 합

↓ Response time : 응답을 시작하는 데 걸리는 시간 처음 반응하는데 까지 걸린 시간

## First-Come First-Serve Scheduling (FCFS)

가장 단순한 **Nonpreemptive(비선점)** CPU 스케줄링 정책이다. 양보X 뺏지 않음

CPU를 먼저 요청한 프로세스가 CPU를 먼저 할당받는다.

**FIFO** 큐로 구현 가능

현재 실행 중인 프로세스가 **자발적으로 실행을 멈추면**, Ready 큐에서 가장 오래 기다린 프로세스가 선택된다.

**waiting time** : Ready Queue에서 CPU를 기다린 시간

**turnaround time** : 도착한 시점부터 작업이 완전히 끝날 때까지 걸린 전체 시간

e.g. P1 -> P2 -> P3

**Waiting time**

Waiting time for P1 = 0; P2 = 24; P3 = 27

Average waiting time:  $(0 + 24 + 27)/3 = 17$

**Turnaround time**

Turnaround time for P1 = 24; P2 = 27; P3 = 30

Average turnaround time:  $(24 + 27 + 30)/3 = 27$

Process Burst Time

P<sub>1</sub> 24

P<sub>2</sub> 3

P<sub>3</sub> 3

Suppose that the processes arrive in the order: P<sub>1</sub>, P<sub>2</sub>, P<sub>3</sub>

The Gantt Chart for the schedule is:



e.g. P2 -> P3 -> P1

**Waiting time**

Waiting time for P1 = 6; P2 = 0; P3 = 3

Average waiting time:  $(6 + 0 + 3)/3 = 3$

**Turnaround time**

Turnaround time for P1 = 30; P2 = 3; P3 = 6

Average turnaround time:  $(30 + 3 + 6)/3 = 13$

Suppose that the processes arrive in the order: P<sub>2</sub>, P<sub>3</sub>, P<sub>1</sub>

The Gantt chart for the schedule is:



**Convoy effect** : 짧은 프로세스가 긴 프로세스 뒤에 서는 현상

→ 하나의 CPU-bound 프로세스와 여러 개의 I/O-bound 프로세스가 있다고 생각해 보자.

| Process | Arrival Time | Service Time (T <sub>s</sub> ) | Start Time | Finish Time | Turnaround Time (T <sub>r</sub> ) | T <sub>r</sub> /T <sub>s</sub> |
|---------|--------------|--------------------------------|------------|-------------|-----------------------------------|--------------------------------|
| W       | 0            | 1                              | 0          | 1           | 1                                 | 1                              |
| X       | 1            | 100                            | 1          | 101         | 100                               | 1                              |
| Y       | 2            | 1                              | 101        | 102         | 100                               | 100                            |
| Z       | 3            | 100                            | 102        | 202         | 199                               | 1.99                           |
| Mean    |              |                                |            |             | 100                               | 26                             |

→ 결론 : FCFS는 CPU bound(CPU를 많이 쓰는)가 많은 경우에 좋음.

## Shortest-Job-First(SJF)

aka = Shortest Process Next(SPN)

**Nonpreemptive(비선점)**

CPU가 available해지면, 다음 CPU burst가 가장 작은 프로세스에게 CPU를 할당

NOT Job이 짧은 P.

두 프로세스의 다음 CPU burst가 같다면 FCFS 방식으로 우선순위를 정한다.

e.g. P4 -> P1 -> P3 -> P2

**Waiting time**

Waiting time for P1 = 3; P2 = 16; P3 = 9; P4 = 0

Average waiting time:  $(3 + 16 + 9 + 0)/4 = 7$

**Turnaround time**

Turnaround time for P1 = 9; P2 = 24; P3 = 16 ; P4 = 3

Average turnaround time:  $(9 + 24 + 16 + 3)/4 = 13$

Process Burst Time

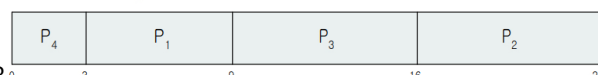
P<sub>1</sub> 6

P<sub>2</sub> 8

P<sub>3</sub> 7

P<sub>4</sub> 3

SJF scheduling chart





SJF 스케줄링 알고리즘은 증명 가능할 정도로 최적이다. **Optimal하다!**

주어진 프로세스 집합에 대해 최소 **평균 대기 시간**을 준다.

짧은 프로세스를 긴 프로세스보다 앞에 두면 → 짧은 프로세스의 waiting time이 줄어든다.

그 감소량이 긴 프로세스의 waiting time 증가량보다 더 크게 작용한다.

결과적으로 **평균 waiting time이 감소한다.**

But... 다음 CPU burst의 길이를 알 방법이 없다.

-> 따라서 각 프로세스에 필요한 처리 시간을 추정한다.

### 다음 CPU burst의 길이 결정

다음 CPU burst의 길이를 예측함. 다음 CPU burst는 이전 CPU burst들과 비슷한 길이일 것이라고 가정.  
예측값(predicted burst)은 있으니까

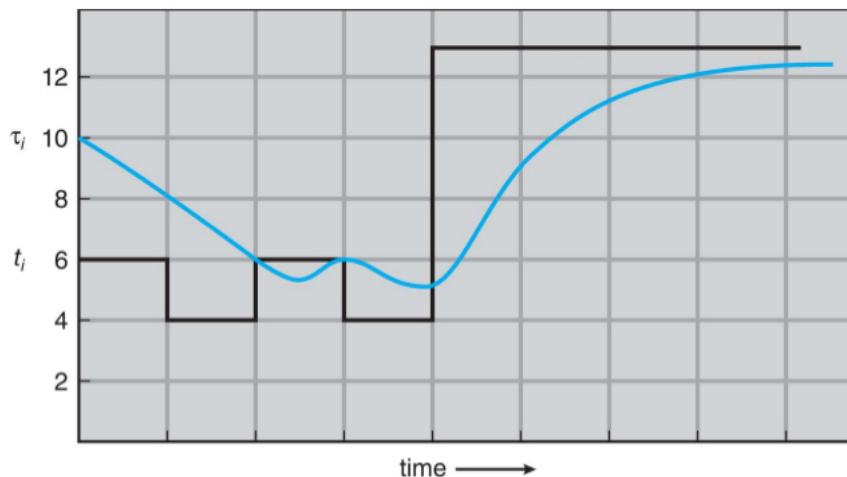
-> 다음 CPU burst는 일반적으로 이전 CPU burst 길이들의 **지수평균(exponential average)**으로 예측한다.

1.  $t_n$  = actual length of  $n^{th}$  CPU burst << n번째 CPU burst의 실제 길이
2.  $\tau_{n+1}$  = predicted value for the next CPU burst << 다음 CPU burst에 대한 예측값
3.  $\alpha, 0 \leq \alpha \leq 1$  << 가중치 알파. 조절을 해야 함.
4. Define:  $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$  << 타우

다음 예측값 = (이번 실제값) + (이전 예측값)을 가중 평균한 것

타우  $\tau$  : 예측값

t = 실측값



|                      |    |   |   |   |    |    |    |     |     |
|----------------------|----|---|---|---|----|----|----|-----|-----|
| CPU burst ( $t_i$ )  | 6  | 4 | 6 | 4 | 13 | 13 | 13 | ... |     |
| "guess" ( $\tau_i$ ) | 10 | 8 | 6 | 6 | 5  | 9  | 11 | 12  | ... |

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n \quad (\alpha = 0.5)$$

$\alpha = 0$

$$\tau_{n+1} = \tau_n$$

<< FCFS와 같은 상황이 일어나게 됨. 최근 실제 기록은 반영되지 않음.

Recent history does not count

$\alpha = 1$

$$\tau_{n+1} = \alpha t_n$$

<< 오직 가장 최근의 실제 CPU burst만 반영된다

Only the actual last CPU burst counts

If we expand the formula, we get:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots + (1 - \alpha)^j \alpha t_{n-j} + \dots + (1 - \alpha)^{n+1} \tau_0$$

$$\tau_{n+1} = 0.8 t_n + 0.16 t_{n-1} + 0.032 t_{n-2} + \dots + (0.2)^{n+1} \tau_0, \quad (\alpha = 0.8)$$



$\alpha$ 와  $(1-\alpha)$ 가 둘 다 1 이하이므로, 뒤로 갈수록 각 항의 가중치는 이전 항보다 더 작아진다  
 -> 오래된 정보일수록 덜 중요하게 본다

$\alpha = 0$

최근 실제값 무시  
 예측 안 바뀜

$\alpha = 1$

가장 최근 실제값만 사용  
 너무 민감함

$0 < \alpha < 1$

최근값과 과거값을 적절히 섞음  
 최근값일수록 더 큰 가중치  
 오래된 값일수록 작은 가중치

### SJF 장점

Response time을 확실하게 줄일 수 있음.

### SJF 단점

긴 프로세스에 대해서 **starvation**이 일어날 수 있음.

Interrupt 기능이 없어서 **time-sharing** 시스템에서 쓸 수 없음

## Shortest Remaining Time First (SRTF)

SJF(SPN)의 preemptive한 버전

SJF(SPN)와 마찬가지로, 처리 시간의 추정이 필요함.

항상 **remaining time**이 가장 짧은 프로세스를 선택함.

### SRTF 장점

더 뛰어난 turnaround time 성능을 제공할 수 있다

실행 중인 긴 작업보다 짧은 작업에 즉시 우선권을 주기 때문이다

-> 도중에 짧은 작업이 들어오면 짧은 작업에게 양보함. 전체 성능 향상

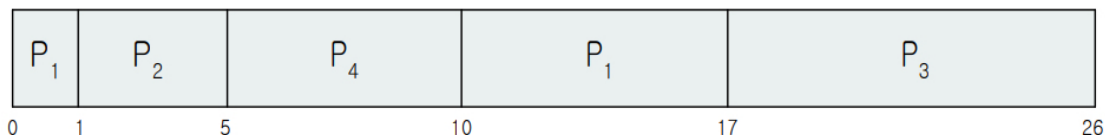
### SRTF 단점

지금까지 실행된 서비스 시간을 기록해야 하므로 **오버헤드가 발생함**

긴 프로세스가 starvation에 빠질 위험이 있다 (SJF와 동일한 단점).

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0            | 8          |
| $P_2$   | 1            | 4          |
| $P_3$   | 2            | 9          |
| $P_4$   | 3            | 5          |

### SRTF Gantt Chart



### Waiting time

Waiting time for  $P_1 = (10-1)$ ;  $P_2 = (1-1)$ ;  $P_3 = (17-2)$ ;  $P_4 = (5-3)$

Average waiting time  $(9+0+15+2)/4 = 26/4 = 6.5$

SJF Average waiting time : 7.75

### Turnaround time

Turnaround time for  $P_1 = (17-0)$ ;  $P_2 = (5-1)$ ;  $P_3 = (26-2)$ ;  $P_4 = (10-3)$

Average turnaround time:  $(17 + 4 + 24 + 7)/4 = 13$

SJF Average turnaround time : 14.25

>>SRTF가 대체적으로 SJF보다 빠르다!

## Round Robin(RR)

FCFS의 Preemptive 버전!

각 프로세스는 CPU 시간을 작은 단위(time quantum 또는 time slice)로 받는데, 보통 10~100ms이다.

Ready Queue는 원형 큐(circular queue)처럼 동작함.

CPU 스케줄러는 Ready Queue를 순서대로 한 바퀴씩 돈다.

-> 각 프로세스에 최대 1 time quantum만큼 CPU를 할당함.

경우 1) burst가 1 time quantum보다 짧은 경우

프로세스가 스스로 CPU를 자발적으로 반납

그 다음 스케줄러는 Ready Queue의 다음 프로세스로 진행

경우 2) CPU burst가 1 time quantum보다 길다면

타이머가 울리고 운영체제에 인터럽트를 발생시킴

context switch가 수행됨

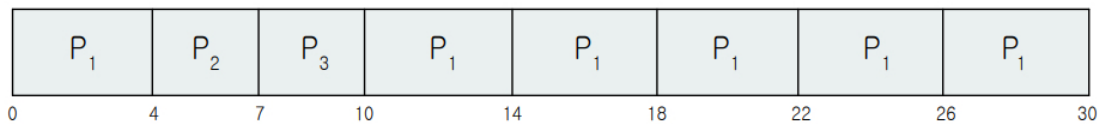
그 프로세스는 Ready Queue의 맨 뒤(tail)로 들어감

그 다음 CPU 스케줄러는 Ready Queue의 다음 프로세스를 선택

e.g. Time Quantum = 4인 RR

| Process | Burst Time |
|---------|------------|
| $P_1$   | 24         |
| $P_2$   | 3          |
| $P_3$   | 3          |

The Gantt chart is:



Waiting time

Waiting time for  $P_1 = (10-4)$ ;  $P_2 = 4$ ;  $P_3 = 7$

Average waiting time:  $(6 + 4 + 7)/3 = 5.66$

SJF Average waiting time : 3

Turnaround time

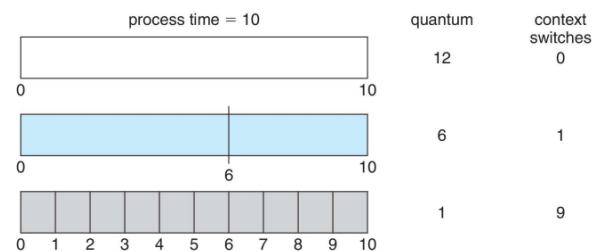
Turnaround time for  $P_1 = 30$ ;  $P_2 = 7$ ;  $P_3 = 10$

Average turnaround time:  $(30 + 7 + 10)/3 = 15.66$

SJF Average turnaround time : 13

Typically, higher average turnaround and waiting time than SJF, but better response

>> SJF보다 turnaround와 waiting time은 길지만, response time은 더 빠르다.



Time Quantum에 따른 RR의 성능

q가 너무 크면

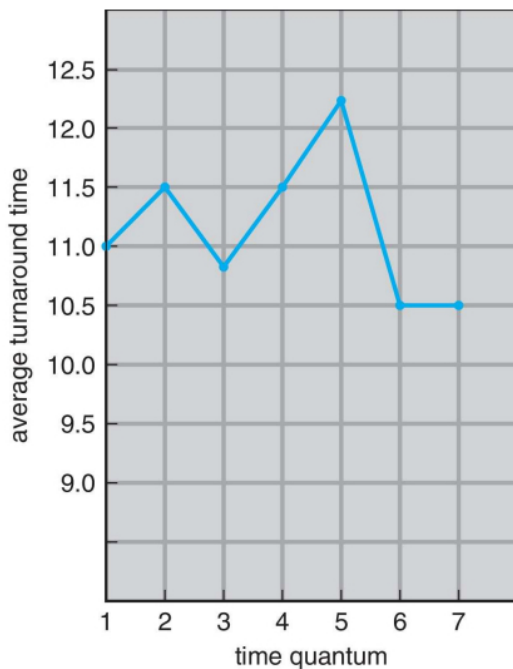
FCFS과 같이 작동함.

q가 너무 작으면

q가 너무 작으면, context switch 시간에 비해 충분히 커야 함

그렇지 않으면 overhead가 너무 커짐

context switch < 10 $\mu$ s 이기 때문에 보통 q를 10ms-100ms로 잡으면 괜찮다!



| process | time |
|---------|------|
| $P_1$   | 6    |
| $P_2$   | 3    |
| $P_3$   | 1    |
| $P_4$   | 7    |

80% of CPU bursts should be **shorter** than  $q$

>>전체 CPU burst를 모아봤을 때, 그 중 80% 정도는  $q$ 보다 CPU burst가 짧아야 한다

### Priority Scheduling

각 프로세스에는 **priority number**가 부여됨.

CPU는 **가장 높은 우선순위를** 가진 프로세스에 할당됨

int 값이 가장 작은 것이 가장 높은 우선순위이다

<- but 구현하기 나름.

만약 우선순위가 같으면, FCFS 순서로 스케줄된다.

SJF 알고리즘은 단순히 하나의 priority 알고리즘 -> 여기서 priority 기준이 짧은 CPU burst

| Process | Burst Time | Priority |
|---------|------------|----------|
| $P_1$   | 10         | 3        |
| $P_2$   | 1          | 1        |
| $P_3$   | 2          | 4        |
| $P_4$   | 1          | 5        |
| $P_5$   | 5          | 2        |

### Priority scheduling Gantt Chart



### Waiting time

Waiting time for  $P_1 = 6$ ;  $P_2 = 0$ ;  $P_3 = 16$ ;  $P_4 = 18$ ;  $P_5 = 1$

Average waiting time:  $(6 + 0 + 16 + 18 + 1)/5 = 8.2$

SJF Average waiting time : 3.2

### Turnaround time

Turnaround time for  $P_1 = 16$ ;  $P_2 = 1$ ;  $P_3 = 18$ ;  $P_4 = 19$ ;  $P_5 = 6$

Average turnaround time:  $(16 + 1 + 18 + 19 + 6)/5 = 12$

SJF Average turnaround time : 7

Priority 방식은 preemptive와 nonpreemptive 둘 다 가능!

Starvation이 가장 큰 문제

일부 낮은 우선순위 프로세스를 무기한 기다리게 만들 수 있음..

Aging : Starvation의 해결책

오래 기다린 프로세스의 priority를 점점 올려주는 방법

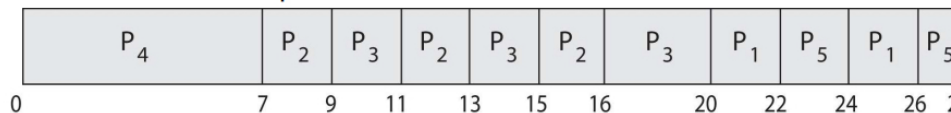
### Priority Scheduling + Round Robin

Priority가 제일 높은 것부터 순서대로 실행

같은 Priority면 Round robin으로 실행.

| Process | Burst Time | Priority |
|---------|------------|----------|
| $P_1$   | 4          | 3        |
| $P_2$   | 5          | 2        |
| $P_3$   | 8          | 2        |
| $P_4$   | 7          | 1        |
| $P_5$   | 3          | 3        |

Gantt Chart with 2 ms time quantum



Waiting time

Waiting time for  $P_1 = (20+2)$ ;  $P_2 = (7+2+2)$ ;  $P_3 = (9+2+1)$ ;  $P_4 = 0$ ;  $P_5 = (22+2)$

Average waiting time:  $(22 + 11 + 12 + 0 + 24)/5 = 13.8$

SJF Average waiting time : 8.2

Turnaround time

Turnaround time for  $P_1 = 26$ ;  $P_2 = 16$ ;  $P_3 = 20$ ;  $P_4 = 7$ ;  $P_5 = 27$

Average turnaround time:  $(26 + 16 + 20 + 7 + 27)/5 = 19.2$

SJF Average turnaround time : 13.6

### Multilevel Queue

priority 스케줄링에서는 각 우선순위마다 별도의 큐를 둘 수 있다

OS는 무조건 Priority가 0인 것부터 봄.

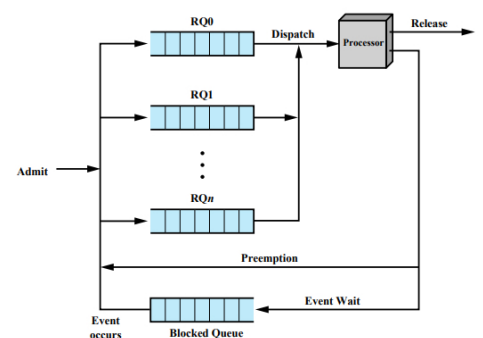
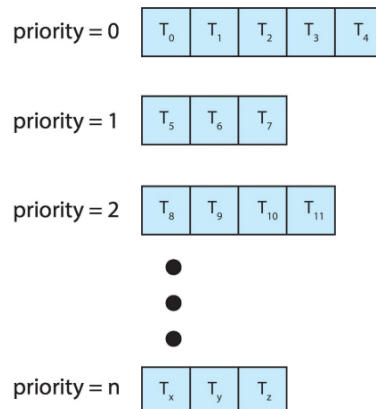


Figure 9.4 Priority Queuing

### Multilevel Feedback Queue

일반 Multilevel Queue에서는 프로세스가 한 큐에 배정되면 거기에 고정되었다.

Multilevel Feedback Queue에서 프로세스는 여러 큐 사이를 이동할 수 있다

다단계 피드백 큐 스케줄러는 다음 매개변수들로 정의된다

큐의 개수

각 큐에서 사용할 스케줄링 알고리즘

프로세스를 언제 상위 큐로 올릴지 결정하는 방법

프로세스를 언제 하위 큐로 내릴지 결정하는 방법

프로세스가 서비스가 필요할 때 어떤 큐로 들어갈지 결정하는 방법

## Three queues:

$Q_0$  – RR with time quantum 8 milliseconds

$Q_1$  – RR time quantum 16 milliseconds

$Q_2$  – FCFS

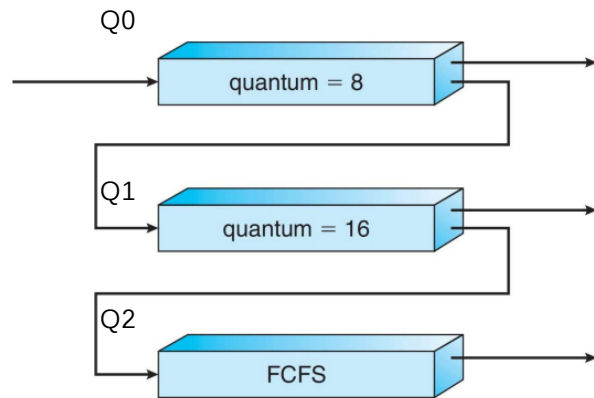
## Scheduling

A new job enters queue  $Q_0$  which is served FCFS

- ▶ When it gains CPU, job receives 8 milliseconds
- ▶ If it does not finish in 8 milliseconds, job is moved to queue  $Q_1$

At  $Q_1$  job is again served FCFS and receives 16 additional milliseconds

- ▶ If it still does not complete, it is preempted and moved to queue  $Q_2$



만약  $Q_0$ 으로 들어간 프로세스가 8ms안에 일을 끝내지 못하면,  $Q_1$ 으로 강등된다.  
강등된 프로세스는  $Q_1$ 에서는 16ms를 받는다.  
그래도 못끝내면  $Q_2$ 로 강등되어서 천천히 작업해라.